

PYTHON FUNCTIONAL PROGRAMMING

THE COMPLETE TEXTBOOK AND PROFESSIONAL REFERENCE

From First-Class Functions to Production Pipelines

map, filter, reduce, closures, decorators, composition, lazy evaluation, itertools, Data Analytics, Data Science, Machine Learning, AI Engineering, real-world projects and interview preparation

Kajola Gbenga
Data Scientist & AI Engineer

Table of Contents

1. Introduction to Functional Programming
2. First-Class Functions
3. Higher-Order Functions
4. Lambda Functions
5. map()
6. filter()
7. reduce()
8. sorted(), key=, and Functional Techniques
9. functools
10. operator Module
11. Partial Functions
12. Closures
13. Immutability and Side Effects
14. Pure Functions
15. Comprehensions as Functional-Style Python
16. Functional Programming with Strings, Lists, Tuples, Sets, Dictionaries
17. Functional Programming for Data Analytics
18. Functional Programming for Data Science and Machine Learning
19. Functional Programming in AI Engineering
20. Functional Programming + Decorators
21. Function Composition
22. Functional Pipelines
23. Lazy Evaluation
24. Functional Programming with itertools
25. Functional Programming vs Traditional Loops
26. Functional Programming and Testing
27. Functional Programming and Concurrency
28. Common Mistakes
29. Professional Best Practices
30. Real-World Projects
31. Interview Preparation
32. Practical Exercises
33. Final Cheat Sheet

1. Introduction to Functional Programming

1.1 What Functional Programming Means

Functional programming (FP) is a programming paradigm that treats computation as the evaluation and composition of functions, rather than as a sequence of instructions that change a program's state step by step. In a functional style, a program is built primarily by combining small, predictable functions that transform data into new data, rather than by mutating variables and objects in place. Python is not a purely functional language, it is a multi-paradigm language, but it provides substantial, genuinely useful functional programming tools, and understanding when and how to use them is an essential skill for writing clean, testable, professional-grade Python, especially in data-heavy fields.

1.2 Imperative vs Procedural vs Object-Oriented vs Functional Programming

Code Line	Explanation
Imperative programming	Describes computation as an explicit sequence of statements that change program state: assign a variable, loop, mutate a list, check a condition. Most beginner Python code (for loops with manual accumulator variables) is written imperatively.
Procedural programming	A specific style of imperative programming organized around procedures (functions) that are called in sequence to perform tasks step by step; still fundamentally about executing ordered instructions and managing state.
Object-oriented programming (OOP)	Organizes code around objects that bundle state (attributes) and behavior (methods) together; computation happens through objects sending messages to each other and mutating their own internal state (already covered in your OOP notes).
Functional programming	Organizes code around the application and composition of functions that transform data into new data, with an emphasis on avoiding mutable state and side effects; a program is closer to a pipeline of transformations than a sequence of state changes.

These are not mutually exclusive categories in Python; a single well-designed Python program routinely mixes all four, using classes to model domain entities (OOP), loops for simple iteration (procedural), and `map()/filter()/comprehensions` for data transformation (functional), choosing whichever style best fits each specific piece of the problem.

1.3 The Core Philosophy of Functional Programming

- Computation as transformation: a program is a series of functions that take input and produce output, rather than a series of steps that mutate shared state.
- Functions as the primary building block: functions are treated as values in their own right, not merely named blocks of instructions (expanded fully in section 2).
- Minimizing mutable state: preferring to create new data over modifying existing data in place, which makes program behavior easier to reason about and predict.
- Declarative over imperative: describing what result is wanted, rather than the exact step-by-step mechanics of how to compute it (expanded in section 1.9).

1.4 Functions as First-Class Objects

The single foundational idea that everything else in this handbook builds on is that, in Python, functions are first-class objects: they can be assigned to variables, passed as arguments, returned from other functions, and stored in data structures, exactly like an integer, a string, or a list. This is covered in complete depth in section 2, but it is worth stating plainly here as the philosophical starting point: functional programming in Python is possible specifically because functions are not a special, restricted kind of thing, they are ordinary values.

1.5 Pure vs Impure Functions

```
# Pure: same input always produces the same output, no observable side effects
def add(a, b):
    return a + b

# Impure: depends on external state, and/or produces a side effect
total = 0
def add_to_total(x):
    global total
    total += x
    return total
```

Code Line	Explanation
Pure function	<code>add(a, b)</code> always returns the same result for the same arguments, and calling it does not change anything outside itself; it has no dependency on, or effect on, any state beyond its own inputs and return value.
Impure function	<code>add_to_total(x)</code> depends on and modifies an external variable (<code>total</code>); calling it twice with the same argument produces two different results, and it has a visible side effect on the program's state beyond its return value.

Pure functions are the ideal building block of functional-style code; a complete treatment of their properties, benefits, and how to design them deliberately is given in section 14.

1.6 Referential Transparency

An expression is referentially transparent if it can be replaced by its resulting value everywhere it appears, without changing the program's behavior in any way. `add(2, 3)` is referentially transparent: it can always be replaced by the literal value 5 with no change in meaning. `add_to_total(2)` is not: replacing it with its return value would silently skip the side effect of updating `total`, changing what the rest of the program does. Referential transparency is a direct consequence of purity (section 1.5) and is a large part of why pure functions are easier to reason about, test, and even automatically optimize.

1.7 Side Effects

A side effect is any observable interaction a function has with the world outside its own local scope and return value: modifying a global variable, mutating a mutable argument passed in, printing to the console, writing to a file, making a network request, or reading from a random number generator or the system clock. Functional programming does not claim side effects should never happen (a program that never interacts with the outside

world is useless), but it emphasizes isolating and minimizing them, keeping the core transformation logic of a program pure and pushing side effects to clearly identified boundaries.

1.8 Immutability

Immutability means data, once created, cannot be changed; any 'modification' instead produces a new piece of data, leaving the original untouched. Functional programming favors immutability because it eliminates an entire category of bugs caused by unexpected mutation, code elsewhere in the program (or even in a different thread) changing a value out from under code that assumed it was stable. Python's own built-in immutable types (str, tuple, frozenset, int, float) already embody this principle; section 13 explores immutability, Python's specific limitations around it, and practical strategies for working with it deliberately.

1.9 Declarative Programming

```
numbers = [1, 2, 3, 4, 5]

# Imperative: describes HOW to compute the result, step by step
squares = []
for n in numbers:
    squares.append(n ** 2)

# Declarative: describes WHAT result is wanted
squares = list(map(lambda n: n ** 2, numbers))
squares = [n ** 2 for n in numbers]
```

Both declarative versions above describe the desired result, a list of squares, without spelling out the mechanics of initializing an empty list and appending to it one element at a time; that mechanical detail is handled internally by `map()` or by the comprehension syntax. This is the essence of declarative style, and functional programming is one major approach to writing declarative code in Python, alongside comprehensions, which are explored specifically as a functional-adjacent style in section 15.

1.10 Advantages of Functional Programming in Python

- Code built from pure functions is easier to test in isolation, since there is no hidden external state to set up or tear down (expanded in section 26).
- Avoiding shared mutable state reduces a large class of bugs related to unexpected changes to data from elsewhere in a program.
- Small, composable functions tend to be more reusable across different parts of a codebase, or across different projects entirely.
- Declarative, transformation-oriented code (`map/filter/comprehensions`) often reads closer to the problem's own description, once the reader is fluent in the style.
- Pure, stateless transformations are naturally well-suited to certain kinds of parallel and distributed processing (with important caveats, covered honestly in section 27).

1.11 Disadvantages and Honest Limitations

- Python was not designed as a purely functional language; some functional idioms (deep recursion, for instance) are noticeably less efficient or less idiomatic in Python than in languages built around FP from the ground up.
- Overusing `lambda`, `reduce()`, or deeply chained functional calls can produce code that is objectively harder to read than an equivalent, well-named loop, especially for other developers less familiar with the style (expanded in sections 28 and 29).
- Python's built-in immutable types are genuinely immutable, but user-defined objects are mutable by default, and truly enforcing immutability across an entire codebase takes deliberate discipline (section 13.7).
- Debugging a long chain of functional transformations can sometimes be harder than stepping through an equivalent imperative loop, since intermediate values are not always bound to a named variable you can inspect.

1.12 When Functional Programming Is Useful

- Data transformation pipelines: cleaning, filtering, mapping, and aggregating records, which map almost directly onto `filter()`, `map()`, and `reduce()`.
- Situations where predictability and testability matter most: financial calculations, validation logic, and anywhere a bug caused by hidden state would be costly.
- Feature engineering and preprocessing steps in Data Science and Machine Learning workflows, which are naturally expressed as a sequence of pure transformations applied to data (fully explored in sections 17 through 19).
- Any situation where a small number of composable, well-tested transformation functions can replace a large, monolithic, stateful block of procedural code.

A Note on This Handbook's Approach: This handbook will not claim functional code is unconditionally 'better' than imperative code. The goal throughout is Pythonic judgment: knowing the functional tools well enough to reach for them exactly when they genuinely improve clarity, correctness, or maintainability, and to avoid them when they do not, a theme returned to explicitly in sections 25, 28, and 29.

2. First-Class Functions

2.1 Functions Are Objects

```
def greet(name):
    return f'Hello, {name}!'

print(type(greet))
print(greet.__name__)
print(id(greet))
```

Code Line	Explanation
<code>type(greet)</code>	Prints <code><class 'function'></code> , confirming that <code>greet</code> is an instance of Python's built-in function type, an object like any other, not a special syntactic construct.
<code>greet.__name__</code>	Every function object carries its own name as an attribute, accessible directly; useful for logging, debugging, and introspection.
<code>id(greet)</code>	Every function has a unique identity in memory, exactly like any other object (connecting directly to the <code>is</code> operator behavior covered in your Operators notes).

2.2 Assigning Functions to Variables

```
def square(x):
    return x * x

my_function = square
print(my_function(5))
print(my_function is square)
```

Code Line	Explanation
<code>my_function = square</code>	Assigns the function object itself (not the result of calling it, note the absence of parentheses) to a new name; <code>my_function</code> and <code>square</code> now both refer to the exact same underlying function object.
<code>my_function(5)</code>	Calling <code>my_function</code> behaves identically to calling <code>square</code> , since they are the same object; produces 25.
<code>my_function is square</code>	Produces <code>True</code> , confirming both names point to the same object in memory, exactly like assigning a list or dictionary to a second variable name.

2.3 Passing Functions as Arguments

```
def apply_twice(func, value):
    return func(func(value))

def increment(x):
    return x + 1

print(apply_twice(increment, 5))
```

`apply_twice()` accepts another function, `increment`, as an ordinary argument, and calls it internally; this is the essential mechanism behind higher-order functions, callbacks, and nearly every functional tool covered in the rest of this handbook, and it is explored formally in section 3.

2.4 Returning Functions From Functions

```
def make_multiplier(factor):
    def multiplier(x):
        return x * factor
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5))
print(triple(5))
```

Code Line	Explanation
<code>return multiplier</code>	<code>make_multiplier()</code> returns the inner function object itself, not the result of calling it; each call to <code>make_multiplier()</code> creates and returns a brand new function object.
<code>double = make_multiplier(2)</code>	<code>double</code> now refers to a specific function that always multiplies by 2, remembering the factor value from when it was created; this remembering behavior is a closure, covered in full depth in section 12.

2.5 Storing Functions in Data Structures

```
operations = {
    'add': lambda a, b: a + b,
    'subtract': lambda a, b: a - b,
    'multiply': lambda a, b: a * b,
}

print(operations['add'](3, 4))
print(operations['multiply'](3, 4))

pipeline = [str.strip, str.lower, str.title]
text = ' hello WORLD '
for step in pipeline:
    text = step(text)
print(text)
```

Code Line	Explanation
<code>operations</code> dictionary	Storing functions as dictionary values creates a dispatch table: a mapping from a name or key directly to the behavior it should trigger, avoiding a long if/elif chain that checks a string and calls a different function in each branch.
<code>pipeline</code> list of functions	A list of function objects (here, unbound string methods) that can be applied in sequence to a value, a simple manual preview of the function composition and pipeline patterns covered fully in sections

Code Line	Explanation
	21 and 22.

2.6 Calling Functions Dynamically

```
def run_operation(name, a, b):
    return operations[name](a, b)

print(run_operation('subtract', 10, 4))
```

Because functions are ordinary values that can be looked up by key, choosing which function to call can itself become a runtime decision driven by data (here, a string name), rather than a hardcoded choice baked into the source code, a pattern used heavily in configuration-driven systems, plugin architectures, and ML pipeline registries (section 21 revisits this idea for pipelines).

2.7 Function Objects and Their Properties

```
def process(a, b=10, *args, **kwargs):
    '''Process two numbers with optional extras.'''
    return a + b

print(process.__name__)
print(process.__doc__)
print(process.__defaults__)
print(process.__code__.co_varnames)
```

Code Line	Explanation
<code>__name__</code>	The function's name as a string.
<code>__doc__</code>	The function's docstring, if one was written directly beneath the <code>def</code> line.
<code>__defaults__</code>	A tuple of the default values for any parameters that have them, here (10,) for the <code>b</code> parameter.
<code>__code__.co_varnames</code>	Exposes low-level details of the function's compiled code object, including the names of its local variables and parameters; rarely needed in everyday code, but useful for building introspection tools, decorators, and testing frameworks.

2.8 Real-World Applications of First-Class Functions

- Callback functions: passing a function to be invoked later, such as an event handler, a completion callback, or a validation hook.
- Strategy selection: storing several interchangeable algorithms as functions in a dictionary or list, and picking one at runtime based on configuration or data.
- Decorators (section 20): a decorator is fundamentally a function that accepts a function and returns a new function, only possible because functions are first-class.
- Building reusable, configurable pipelines (sections 21-22): a pipeline is, at its core, an ordered collection of function objects applied in sequence.

3. Higher-Order Functions

3.1 Definition

A higher-order function is any function that does at least one of the following: accepts one or more functions as arguments, returns a function as its result, or both. This is a direct, practical consequence of functions being first-class objects (section 2); higher-order functions are simply ordinary functions that happen to treat other functions as data.

3.2 Functions That Accept Functions

```
def apply_to_all(func, items):
    return [func(item) for item in items]

def celsius_to_fahrenheit(c):
    return c * 9 / 5 + 32

temps = [0, 20, 37, 100]
print(apply_to_all(celsius_to_fahrenheit, temps))
```

`apply_to_all()` is a higher-order function: its behavior is customized entirely by which function is passed in as `func`, making it a general-purpose transformation tool rather than logic specific to temperature conversion. Built-in examples of this same pattern include `map()`, `filter()`, and `sorted(key=...)`, covered in sections 5, 6, and 8.

3.3 Functions That Return Functions

```
def make_validator(min_value, max_value):
    def validator(value):
        return min_value <= value <= max_value
    return validator

is_valid_age = make_validator(0, 120)
is_valid_percentage = make_validator(0, 100)

print(is_valid_age(45))
print(is_valid_percentage(150))
```

`make_validator()` is a higher-order function factory: it produces new, specialized functions on demand, each remembering its own `min_value` and `max_value`. This pattern, generating specialized functions from a general template, is explored further as closures in section 12 and as partial application in section 11.

3.4 A Practical Example: Combining Both Directions

```
def logged(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__} with {args}')
        result = func(*args, **kwargs)
        print(f'{func.__name__} returned {result}')
        return result
    return wrapper
```

```
def add(a, b):
    return a + b

logged_add = logged(add)
logged_add(3, 4)
```

Code Line	Explanation
logged(func)	Accepts a function (add) as an argument, making it higher-order in the 'accepts a function' sense.
return wrapper	Returns a brand new function (wrapper), making it higher-order in the 'returns a function' sense too.
Why this matters	This exact pattern, a higher-order function that both accepts and returns a function, is precisely what a decorator is; section 20 formalizes this connection completely.

3.5 Higher-Order Functions vs Ordinary Functions

Code Line	Explanation
Ordinary function	Operates directly on data: numbers, strings, lists. Its behavior is fixed once defined.
Higher-order function	Operates on behavior itself, by accepting and/or producing functions. Its concrete behavior can be customized or specialized at the point it is used, without modifying its own source code.

3.6 The Relationship Between Higher-Order Functions and Decorators

A decorator, using Python's `@decorator_name` syntax, is syntactic sugar for exactly the pattern shown in section 3.4: taking a function and replacing it with a new function that wraps it with additional behavior. Every decorator is a higher-order function, though not every higher-order function is used as a decorator. Because your existing notes may not have covered this connection in depth, section 20 revisits decorators specifically through this functional-programming lens, building progressively from first-class functions, through higher-order functions and closures, to a complete understanding of how `@decorator` syntax works underneath.

4. Lambda Functions

4.1 Syntax

```
lambda parameters: expression
```

Code Line	Explanation
lambda	The keyword that begins an anonymous (unnamed) function expression.
parameters	A comma-separated list of parameter names, exactly like a def function's parameter list, but without parentheses around them and without type annotations or default-heavy syntax typically used.
expression	A single expression whose evaluated result becomes the lambda's return value automatically; there is no explicit return keyword, and there cannot be multiple statements.

4.2 A Basic Example

```
square = lambda x: x ** 2
print(square(5))

add = lambda a, b: a + b
print(add(3, 4))
```

Assigning a lambda to a variable, as shown here, creates something that behaves like a named function, but this pattern is generally discouraged in favor of `def` (see section 4.14); lambdas are intended primarily for short, throwaway functions passed directly as arguments, as shown throughout the rest of this section.

4.3 Parameters and Return Values

```
no_args = lambda: 'no parameters'
one_arg = lambda x: x * 2
multi_args = lambda x, y, z: x + y + z
with_default = lambda x, y=10: x + y
```

Lambdas support the same parameter features as ordinary functions: zero or more positional parameters, and default values. What they do not support is `*args` and `**kwargs` style flexibility being commonly used in practice (though technically permitted syntactically) or any statement beyond a single returned expression.

4.4 The Single-Expression Limitation

```
# Invalid: a lambda cannot contain statements like if/else blocks or assignments
# broken = lambda x: if x > 0: return 'positive' # SyntaxError

# Valid: a conditional EXPRESSION is allowed, since it evaluates to a single value
classify = lambda x: 'positive' if x > 0 else 'negative' if x < 0 else 'zero'
print(classify(-5))
```

A lambda body must be exactly one expression; it cannot contain an if statement, a for loop, a with block, multiple statements separated by newlines, or an assignment. A conditional expression (the ternary a if condition else b form) is allowed because it is itself a single expression that evaluates to one value, not a full if statement.

4.5 Lambda vs def

Code Line	Explanation
lambda	Anonymous, limited to a single expression, typically used inline as an argument to another function, has no natural docstring or clean stack-trace name (shown as '<lambda>' in a traceback).
def	Named, supports any number of statements, loops, conditionals, docstrings, and complex logic; produces clearer tracebacks and is reusable by name across the codebase.
Key Clarification: A lambda is not a faster or more 'functional' version of a def function; it is simply a more compact syntax for a very short, single-expression function used inline. Anything a lambda can do, an equivalent def function can also do, usually more readably for anything beyond a trivial expression.	

4.6 Lambda with map()

```
numbers = [1, 2, 3, 4]
squared = list(map(lambda x: x ** 2, numbers))
print(squared)
```

Passing a lambda directly as map()'s first argument is one of the most common and idiomatic lambda use cases, since the transformation is typically small enough to fit in one expression and is used only at this single call site. map() itself is covered in full depth in section 5.

4.7 Lambda with filter()

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

Exactly the same pattern applied to filter() (covered fully in section 6): the lambda expresses the keep/discard condition concisely, inline, at the point it is used.

4.8 Lambda with sorted()

```
words = ['banana', 'kiwi', 'apple', 'fig']
by_length = sorted(words, key=lambda w: len(w))
print(by_length)
```

key=lambda w: len(w) tells sorted() to rank each element by its length rather than its natural (alphabetical) ordering; this key= pattern, and when to prefer operator.itemgetter or operator.attrgetter instead of a lambda, is covered comprehensively in section 8.

4.9 Lambda with min() and max()

```
students = [('Ada', 92), ('Grace', 88), ('Alan', 95)]
top_student = max(students, key=lambda s: s[1])
```

```
print(top_student)
```

`min()` and `max()` both accept the same `key=` argument as `sorted()`, letting a lambda define exactly what 'largest' or 'smallest' means for complex data like tuples or dictionaries, here selecting the student tuple with the highest second element (score).

4.10 Lambda with Dictionaries

```
scores = {'Ada': 92, 'Grace': 88, 'Alan': 95}
top_name = max(scores, key=lambda name: scores[name])
print(top_name)

sorted_items = sorted(scores.items(), key=lambda item: item[1], reverse=True)
print(sorted_items)
```

The first example ranks dictionary keys by looking up their associated value inside the lambda; the second sorts (key, value) tuples produced by `.items()` by their second element, a very common pattern for ranking dictionary contents by value.

4.11 Lambda with Tuples

```
points = [(1, 5), (3, 2), (0, 8)]
by_y = sorted(points, key=lambda point: point[1])
print(by_y)
```

Indexing into a tuple inside a lambda (`point[1]`) is a common, direct way to sort or rank tuple-based records by a specific field position; section 8.7 covers `operator.itemgetter` as a more readable alternative for exactly this pattern.

4.12 Multiple Arguments

```
from functools import reduce
combine = lambda acc, x: acc + x
total = reduce(combine, [1, 2, 3, 4], 0)
print(total)
```

A lambda used with `reduce()` (section 7) typically takes exactly two parameters: the running accumulator and the next item, exactly like the `combine` lambda above.

4.13 Nested Lambdas

```
make_adder = lambda n: lambda x: x + n
add_five = make_adder(5)
print(add_five(10))
```

A lambda can itself return another lambda, which is functionally identical to the nested `def` pattern from section 2.4, just written more compactly. This works, but nested lambdas are a clear example of the readability tradeoff discussed in section 4.14; most developers find `make_adder = lambda n: lambda x: x + n` noticeably harder to parse at a glance than the equivalent nested `def` version.

4.14 When Lambda Improves Readability

- The function is genuinely trivial (a single small expression) and used exactly once, inline, as an argument.
- Naming the function separately would add more visual noise than it removes, since the lambda's purpose is obvious from its immediate context (for example, `key=lambda x: x[1]` right next to a `sorted()` call).

4.15 When Lambda Should NOT Be Used

- The logic requires more than one expression, multiple statements, loops, or exception handling; use `def` instead.
- The function is reused in multiple places; give it a real name with `def` so it can be referenced, documented, and tested properly.
- The lambda is assigned to a variable purely to give it a name (`x = lambda a, b: a + b`); this is explicitly discouraged by Python's own style guide (PEP 8) in favor of `def x(a, b): return a + b`, which produces clearer tracebacks and supports a docstring.
- The lambda is deeply nested or combined with several other lambdas in one line, producing code that is difficult to read at a glance (see section 28.1 for a detailed example of this specific mistake).

4.16 Common Beginner Mistakes

```
# Mistake 1: trying to use a statement inside a lambda
# broken = lambda x: print(x); return x # SyntaxError

# Mistake 2: forgetting a lambda always needs to be called to get its result
result = (lambda x: x * 2) # this is a function object, not 6
print(result)
result = (lambda x: x * 2)(3) # correctly called immediately
print(result)

# Mistake 3: overcomplicating a lambda instead of just using def
process = lambda x: x ** 2 if x > 0 else (0 if x == 0 else -x ** 2)
```

Code Line	Explanation
Mistake 1	A lambda cannot contain a semicolon-separated sequence of statements; it is one expression only.
Mistake 2	Defining a lambda without calling it produces the function object itself, not a computed value; the trailing (3) is required to actually invoke it.
Mistake 3	Nested ternary conditional expressions inside a lambda quickly become difficult to read; this specific example is a strong candidate for a properly named <code>def</code> function with <code>if/elif/else</code> instead.

5. map()

5.1 Syntax

```
map(function, iterable, *more_iterables)
```

map() applies function to every element of iterable in turn, and returns a map object, a lazy iterator that produces the transformed values one at a time as they are requested, rather than computing and storing them all immediately (fully explained in section 5.11 and again in section 23).

5.2 How map() Works

```
numbers = [1, 2, 3, 4]
result = map(lambda x: x * 2, numbers)
print(result)
print(list(result))
```

Code Line	Explanation
print(result)	Prints something like <map object at 0x...>, not the transformed values themselves, because map() has not actually computed anything yet; it has only set up the transformation to happen on demand.
list(result)	Forces map() to actually iterate through every element, applying the function to each and collecting the results into a concrete list: [2, 4, 6, 8].

5.3 Mapping One Iterable

```
names = ['ada', 'grace', 'alan']
titled = list(map(str.title, names))
print(titled)
```

Passing an unbound method like str.title directly (without calling it) works because map() itself calls function(item) for each item; str.title(item) is exactly equivalent to item.title().

5.4 Mapping Multiple Iterables

```
prices = [10, 20, 30]
quantities = [2, 3, 1]
totals = list(map(lambda p, q: p * q, prices, quantities))
print(totals)
```

Code Line	Explanation
map(lambda p, q: p * q, prices, quantities)	When more than one iterable is passed, map() calls the function with one corresponding element from each iterable at every step; the function here must accept exactly as many parameters as there are iterables (two, matching prices and quantities).
Length handling	map() stops as soon as the shortest input iterable is exhausted; if prices and quantities had different lengths, the extra elements in the longer one would simply be ignored, with no error raised.

5.5 map() With Named Functions

```
def fahrenheit_to_celsius(f):
    return (f - 32) * 5 / 9

temps_f = [32, 68, 100, 212]
temps_c = list(map(fahrenheit_to_celsius, temps_f))
print(temps_c)
```

A named function is generally preferable to a lambda inside `map()` whenever the transformation has real meaning, is reused elsewhere, or is complex enough to benefit from a docstring, directly following the lambda-vs-def guidance from section 4.15.

5.6 map() With Strings

```
words = ['hello', 'world', 'python']
lengths = list(map(len, words))
print(lengths)

csv_line = '1,2,3,4,5'
numbers = list(map(int, csv_line.split(',')))
print(numbers)
```

The second example is an extremely common real-world pattern: splitting a delimited text line into string pieces, then mapping `int` (or `float`) over the pieces to convert every one to a number in a single, concise line, avoiding a manual loop with an accumulator list.

5.7 map() With Numbers

```
values = [4, 9, 16, 25]
roots = list(map(lambda x: x ** 0.5, values))
print(roots)
```

5.8 map() With Dictionaries

```
records = [{'name': 'Ada', 'score': 92}, {'name': 'Alan', 'score': 78}]
names = list(map(lambda r: r['name'], records))
print(names)

boosted = list(map(lambda r: {**r, 'score': r['score'] + 5}, records))
print(boosted)
```

Code Line	Explanation
<code>lambda r: r['name']</code>	Extracts a single field from every dictionary in a list of records, a very common data-extraction pattern.
<code>{**r, 'score': r['score'] + 5}</code>	The <code>**</code> unpacking syntax builds a new dictionary containing all of <code>r</code> 's existing key-value pairs, then overrides just the <code>score</code> key with an updated value; this creates a new dictionary rather than mutating the original <code>r</code> in place, in keeping with the immutability preference introduced in section 1.8.

5.9 map() With Realistic Datasets

```
transactions = [
    {'id': 1, 'amount': 100.0, 'currency': 'USD'},
    {'id': 2, 'amount': 85.5, 'currency': 'USD'},
]

def apply_tax(txn, rate=0.08):
    return {**txn, 'amount_with_tax': round(txn['amount'] * (1 + rate), 2)}

taxed = list(map(apply_tax, transactions))
print(taxed)
```

This pattern, mapping a named transformation function over a list of record dictionaries, is the direct foundation for the Data Analytics and Data Science examples covered fully in sections 17 and 18.

5.10 Converting map Objects to Other Collections

```
numbers = [1, 2, 3]
as_list = list(map(lambda x: x * 2, numbers))
as_set = set(map(lambda x: x * 2, numbers))
as_tuple = tuple(map(lambda x: x * 2, numbers))
```

A map object is a one-time-use iterator; wrapping it in `list()`, `set()`, or `tuple()` consumes it fully and produces a genuine, reusable collection of that type.

5.11 Lazy Evaluation

```
def noisy_double(x):
    print(f'Processing {x}')
    return x * 2

result = map(noisy_double, [1, 2, 3])
print('map() created, nothing printed yet')
list(result)
```

Nothing is printed until `list(result)` actually iterates through the map object; `map()` itself only records what transformation to apply and to what, without doing any work upfront. This laziness means `map()` can, in principle, work with an infinite or very large stream of data without ever needing to hold the whole thing in memory, a property explored fully and formally in section 23.

5.12 map() vs List Comprehensions

```
numbers = [1, 2, 3, 4]
a = list(map(lambda x: x ** 2, numbers))
b = [x ** 2 for x in numbers]
```

Code Line	Explanation
Equivalence	Both lines produce the exact same result, <code>[1, 4, 9, 16]</code> ; a list comprehension is, in effect, Python's own built-in, often more readable, syntax for exactly this map-and-collect pattern.
When <code>map()</code> reads better	When the transformation function already exists and has a meaningful

Code Line	Explanation
	name (map(str.title, names) versus [name.title() for name in names], arguably a toss-up, but map() avoids re-typing the variable).
When a comprehension reads better	When the transformation is a short expression that would otherwise need an inline lambda; [x ** 2 for x in numbers] is generally considered more directly readable than list(map(lambda x: x ** 2, numbers)).

Section 15 covers comprehensions in their own complete depth, including this comparison from the comprehension side, and section 25 revisits the map()-vs-comprehension-vs-loop choice with a broader, side-by-side comparison across all three styles.

5.13 Performance Considerations

map() with a built-in function (like str.title or int) is typically implemented efficiently and can be marginally faster than an equivalent comprehension, since it avoids the overhead of repeatedly calling into a Python-level lambda for each element. With a lambda, however, the performance difference between map() and an equivalent comprehension is usually negligible, and readability should be the deciding factor rather than a speed difference that rarely matters outside of tight, performance-critical loops.

6. filter()

6.1 Syntax

```
filter(function, iterable)
```

`filter()` returns a lazy iterator (a filter object) containing only the elements of `iterable` for which `function(element)` is truthy. Unlike `map()`, `filter()` always works with exactly one iterable and never transforms the surviving elements, it only decides whether each one is kept or discarded.

6.2 Filtering With Lambda

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)
```

6.3 Filtering With Named Functions

```
def is_valid_email(text):
    return '@' in text and '.' in text.split('@')[-1]

emails = ['ada@example.com', 'not-an-email', 'grace@python.org']
valid = list(filter(is_valid_email, emails))
print(valid)
```

As with `map()`, a named function is preferable when the filtering condition has real business meaning worth documenting and potentially reusing elsewhere, rather than being a trivial one-line comparison.

6.4 Filtering Numbers

```
scores = [55, 72, 88, 45, 91, 60]
passing = list(filter(lambda s: s >= 60, scores))
print(passing)
```

6.5 Filtering Strings

```
words = ['apple', '', 'banana', ' ', 'cherry', None]
clean = list(filter(lambda w: w and w.strip(), words))
print(clean)
```

`w` and `w.strip()` first checks that `w` is truthy (not `None` and not an empty string), short-circuiting before calling `.strip()` on it, then confirms the stripped text is non-empty too, safely filtering out `None`, empty strings, and whitespace-only strings in one combined condition (directly applying the short-circuit evaluation logic from your Operators notes).

6.6 Filtering Dictionaries

```
inventory = {'apples': 0, 'bananas': 12, 'cherries': 0, 'dates': 5}
in_stock = dict(filter(lambda item: item[1] > 0, inventory.items()))
```

```
print(in_stock)
```

Code Line	Explanation
<code>inventory.items()</code>	Produces an iterable of (key, value) tuples, which <code>filter()</code> can process; <code>item[1]</code> accesses the value half of each tuple to check the condition.
<code>dict(filter(...))</code>	Wrapping the filtered (key, value) pairs back in <code>dict()</code> reconstructs a proper dictionary containing only the entries that passed the condition.

6.7 Filtering Records

```
customers = [
    {'name': 'Ada', 'active': True, 'spend': 1200},
    {'name': 'Alan', 'active': False, 'spend': 300},
    {'name': 'Grace', 'active': True, 'spend': 950},
]

high_value_active = list(filter(
    lambda c: c['active'] and c['spend'] > 500, customers
))
print(high_value_active)
```

Filtering a list of record dictionaries by a compound business condition is one of the most practically common uses of `filter()` in real Data Analytics work, directly previewing the fuller examples in section 17.

6.8 Truthy/Falsy Filtering

```
values = [0, 1, '', 'text', None, [], [1, 2], False, True]
truthy_only = list(filter(None, values))
print(truthy_only)
```

Code Line	Explanation
<code>filter(None, iterable)</code>	A special, idiomatic case: passing <code>None</code> instead of a function tells <code>filter()</code> to keep only the elements that are themselves truthy (directly reusing the truthy/falsy rules from your Operators notes), without needing to write <code>lambda x: x</code> explicitly.
Result	<code>[1, 'text', [1, 2], True]</code> ; every falsy value (<code>0</code> , <code>''</code> , <code>None</code> , <code>[]</code> , <code>False</code>) is removed.

6.9 filter() vs List Comprehensions

```
numbers = [1, 2, 3, 4, 5, 6]
a = list(filter(lambda x: x % 2 == 0, numbers))
b = [x for x in numbers if x % 2 == 0]
```

Both produce `[2, 4, 6]`; as with `map()`, a comprehension with an `if` clause is often considered more directly readable than `filter()` combined with a `lambda`, especially once the condition is more than a very simple check, while `filter(None, ...)` and `filter(named_function, ...)` remain genuinely idiomatic and clear on their own.

6.10 Practical Data-Cleaning Examples

```
raw_rows = [
    {'id': 1, 'value': 42.5},
    {'id': 2, 'value': None},
    {'id': 3, 'value': float('nan')},
    {'id': 4, 'value': -1},
]

import math
clean_rows = list(filter(
    lambda r: r['value'] is not None
    and not (isinstance(r['value'], float) and math.isnan(r['value']))
    and r['value'] >= 0,
    raw_rows
))
print(clean_rows)
```

A realistic multi-condition data-cleaning filter: rejecting missing values (None), rejecting NaN (not-a-number floating point markers, common in numeric data pipelines), and rejecting negative values, all combined into a single filter() call, directly foreshadowing the fuller data-cleaning pipeline examples built in sections 17 and 22.

7. reduce()

7.1 Why reduce() Is Not a Built-in

Unlike `map()` and `filter()`, `reduce()` lives in the `functools` module and must be explicitly imported. This was a deliberate design decision: Guido van Rossum, Python's creator, has explained that `reduce()` is often significantly harder to read at a glance than an equivalent explicit loop, especially for anyone unfamiliar with the accumulator concept, and moving it out of the built-in namespace nudges developers toward considering whether a loop, or one of Python's more specific aggregation built-ins (`sum()`, `min()`, `max()`, `any()`, `all()`), would communicate intent more clearly.

7.2 Syntax

```
from functools import reduce
reduce(function, iterable, initializer=None)
```

Code Line	Explanation
function	A function that accepts exactly two arguments: the accumulated result so far, and the next element from the iterable; it must return the new accumulated result.
iterable	The sequence of values to reduce down to a single combined result.
initializer (optional)	A starting value for the accumulator; if omitted, the iterable's first element is used as the initial accumulator value, and reduction starts from the second element.

7.3 The Accumulator Concept

`reduce()` works by maintaining a single running value, the accumulator, that is updated one element at a time: the function is called with the current accumulator and the next item, and its return value becomes the new accumulator for the next step. After processing every element, the final accumulator value is `reduce()`'s overall result.

7.4 Step-by-Step Execution

```
from functools import reduce
numbers = [1, 2, 3, 4]
total = reduce(lambda acc, x: acc + x, numbers)
print(total)
```

Code Line	Explanation
Step 1	No initializer given, so <code>acc</code> starts as the first element, 1; the next element, <code>x</code> , is 2. <code>lambda(1, 2)</code> returns 3.
Step 2	<code>acc</code> is now 3 (the previous result); <code>x</code> is 3 (the next element). <code>lambda(3, 3)</code> returns 6.
Step 3	<code>acc</code> is now 6; <code>x</code> is 4 (the last element). <code>lambda(6, 4)</code> returns 10.
Final result	10, with no more elements left to process; this becomes <code>reduce()</code> 's return value.

7.5 Sum and Product

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
total = reduce(lambda acc, x: acc + x, numbers, 0)
product = reduce(lambda acc, x: acc * x, numbers, 1)
print(total, product)
```

An explicit initializer (0 for a sum, 1 for a product) is good practice: it guarantees a sensible, predictable result even if `numbers` happens to be empty (which would otherwise raise a `TypeError`, since `reduce()` cannot pick a first element from nothing without an initializer).

7.6 Maximum and Minimum

```
from functools import reduce
numbers = [4, 9, 2, 7, 5]
maximum = reduce(lambda acc, x: x if x > acc else acc, numbers)
print(maximum)
```

This works, but Python's own built-in `max()` (covered in section 4.9) is the correct, idiomatic tool for this specific task; this example exists specifically to illustrate `reduce()`'s mechanics using a familiar operation, not to recommend `reduce()` over `max()` in real code (revisited explicitly in section 7.11).

7.7 String Operations

```
from functools import reduce
words = ['Python', 'is', 'fun']
sentence = reduce(lambda acc, w: acc + ' ' + w, words)
print(sentence)
```

Again, this works, but `''.join(words)` is the correct, idiomatic, and significantly faster tool for joining strings; this example is included purely to demonstrate `reduce()`'s general applicability to non-numeric accumulation, not as a recommended pattern.

7.8 Combining Records

```
from functools import reduce
orders = [
    {'category': 'Electronics', 'total': 250},
    {'category': 'Books', 'total': 40},
    {'category': 'Electronics', 'total': 100},
]

totals_by_category = reduce(
    lambda acc, order: {**acc, order['category']: acc.get(order['category'], 0) +
                        order['total']},
    orders,
    {}
)
print(totals_by_category)
```

A genuinely practical use of `reduce()`: building up a summary dictionary (total spend per category) by folding a list of records into a single accumulated result, a pattern that `map()` and `filter()` alone cannot express, since the result's shape (a dictionary) is fundamentally different from the input's shape (a list).

7.9 reduce() vs Loops

```
# reduce()
from functools import reduce
total = reduce(lambda acc, x: acc + x, numbers, 0)

# equivalent loop
total = 0
for x in numbers:
    total = total + x
```

Both produce an identical result; the loop version is generally considered more approachable and immediately readable, especially to developers less fluent in the accumulator/reduce style, while the `reduce()` version is more compact and, once its shape is familiar, expresses the 'fold a sequence down to one value' intent very directly.

7.10 reduce() vs sum(), min(), and max()

Code Line	Explanation
<code>sum(numbers)</code>	The correct, idiomatic, and faster choice for simple summation; never use <code>reduce()</code> for a plain sum in real code.
<code>min(numbers) / max(numbers)</code>	The correct, idiomatic, and faster choice for finding extremes; never use <code>reduce()</code> for this in real code.
<code>reduce()</code>	Reserved for genuine accumulation logic that Python's specific built-ins do not already cover directly, such as the category-totals dictionary example in section 7.8, or combining records into a custom accumulated structure.

7.11 When reduce() Is Appropriate

- The accumulation logic produces something other than a single number or a simple concatenated string, such as building a dictionary, a custom object, or a nested structure from a sequence of records.
- No existing built-in (`sum`, `min`, `max`, `any`, `all`, `".join"`) already expresses the exact same operation more directly.
- The reduction step's logic is simple enough to remain readable as a single lambda or a clearly named helper function.

7.12 When reduce() Hurts Readability

- A specific built-in (`sum`, `min`, `max`) already does exactly what is needed; using `reduce()` instead only adds an unnecessary layer of indirection.
- The accumulator lambda's logic is itself complex, nested, or hard to follow at a glance; in this case, a named function passed to `reduce()`, or simply a plain loop, is clearer.

- A teammate unfamiliar with `reduce()`'s accumulator mechanics would need to mentally trace through the step-by-step execution (section 7.4) just to understand what a single line of code does; this cost is often not worth `reduce()`'s compactness (this exact tradeoff is revisited in section 28.2).

8. sorted(), key=, and Functional Techniques

8.1 sorted()

```
numbers = [5, 2, 8, 1, 9]
print(sorted(numbers))
print(sorted(numbers, reverse=True))
```

`sorted()` is itself a small example of a higher-order function pattern: it accepts an optional `key=` function argument (section 8.2) that customizes how elements are compared, without `sorted()` itself needing to know anything about the data's structure in advance.

8.2 The `key=` Argument

```
words = ['banana', 'kiwi', 'fig', 'apple']
print(sorted(words, key=len))
```

Code Line	Explanation
<code>key=len</code>	Instead of comparing the words directly (which would sort alphabetically), <code>sorted()</code> calls <code>len(word)</code> for every element first, and sorts based on those computed lengths instead; <code>key=</code> is called exactly once per element, not repeatedly during comparisons, which keeps it efficient even for an expensive key function.

8.3 Lambda Expressions as Keys

```
words = ['banana', 'kiwi', 'fig', 'apple']
print(sorted(words, key=lambda w: w[-1]))
```

Sorts words by their last character, a transformation too specific to have a matching built-in function, making a lambda the natural, idiomatic choice here.

8.4 Sorting Dictionaries

```
scores = {'Ada': 92, 'Grace': 88, 'Alan': 95}
by_score_desc = sorted(scores.items(), key=lambda item: item[1], reverse=True)
print(by_score_desc)
```

8.5 Sorting Lists of Tuples

```
points = [(3, 1), (1, 5), (2, 2)]
sorted_by_x = sorted(points, key=lambda p: p[0])
sorted_by_y = sorted(points, key=lambda p: p[1])
```

8.6 Sorting Lists of Dictionaries

```
employees = [
    {'name': 'Ada', 'department': 'Engineering', 'salary': 95000},
    {'name': 'Alan', 'department': 'Sales', 'salary': 72000},
```

```
{'name': 'Grace', 'department': 'Engineering', 'salary': 88000},
]
by_salary = sorted(employees, key=lambda e: e['salary'], reverse=True)
```

8.7 Multiple-Level Sorting

```
by_dept_then_salary = sorted(
    employees,
    key=lambda e: (e['department'], -e['salary'])
)
```

Code Line	Explanation
key=lambda e: (e['department'], - e['salary'])	Returning a tuple from the key function sorts primarily by the tuple's first element (department, alphabetically), then breaks any tie using the second element (salary); negating salary flips just that one field to descending order while department stays ascending, a common technique for mixed-direction multi-level sorts within a single sorted() call.

8.8 operator.itemgetter()

```
from operator import itemgetter
employees_sorted = sorted(employees, key=itemgetter('salary'), reverse=True)
points_sorted = sorted([(3, 1), (1, 5)], key=itemgetter(1))
```

Code Line	Explanation
itemgetter('salary')	Produces a small callable equivalent to lambda e: e['salary'], but implemented in fast C code and generally considered more readable and idiomatic than an equivalent lambda for this specific, extremely common 'get one field' pattern.
itemgetter(1)	Works identically for sequences (tuples, lists) using a numeric index, equivalent to lambda p: p[1].
itemgetter('dept', 'salary')	Passing multiple keys/indices returns a tuple of all of them at once, directly replacing key=lambda e: (e['dept'], e['salary']) for multi-level sorting.

8.9 operator.attrgetter()

```
from operator import attrgetter

class Employee:
    def __init__(self, name, salary):
        self.name, self.salary = name, salary

people = [Employee('Ada', 95000), Employee('Alan', 72000)]
by_salary = sorted(people, key=attrgetter('salary'))
```

attrgetter('salary') is the object-attribute equivalent of itemgetter, producing a callable equivalent to lambda e: e.salary, used when sorting a list of objects (instances of a class) rather than dictionaries or tuples.

8.10 operator.methodcaller()

```
from operator import methodcaller
words = ['Banana', 'apple', 'Cherry']
sorted_ci = sorted(words, key=methodcaller('lower'))
print(sorted_ci)
```

`methodcaller('lower')` produces a callable equivalent to `lambda w: w.lower()`, calling a named method (with no arguments here, though arguments can be supplied too) on each element; useful for case-insensitive sorting and similar method-based key transformations.

8.11 Choosing Between Lambda and operator Functions

Code Line	Explanation
Prefer <code>operator.itemgetter/attrgetter</code> <code>/methodcaller</code>	When the key is a direct field, index, or no-argument method access with no additional computation; these are typically faster (implemented in C) and instantly recognizable in intent.
Prefer a <code>lambda</code>	When the key involves any actual computation, combination of fields, or conditional logic beyond a direct single-field access, since <code>itemgetter/attrgetter</code> cannot express that.

Section 10 covers the `operator` module in complete depth beyond just these sorting helpers, including its full set of functional arithmetic and comparison tools.

9. functools

The functools module is the standard library's dedicated home for higher-order functions and tools that operate on or produce other functions. This section covers its most important members in depth, several of which are referenced elsewhere in this handbook but explained fully here.

9.1 reduce()

Covered in complete depth in section 7; included here only for completeness of the functools reference.

9.2 partial()

```
from functools import partial

def power(base, exponent):
    return base ** exponent

square = partial(power, exponent=2)
cube = partial(power, exponent=3)
print(square(5), cube(5))
```

partial() creates a new callable with some arguments already 'locked in'. This is covered in full, dedicated depth with many more examples in section 11, since it is significant enough to warrant its own complete treatment.

9.3 partialmethod()

```
from functools import partialmethod

class TextProcessor:
    def transform(self, text, mode):
        return text.upper() if mode == 'upper' else text.lower()

    shout = partialmethod(transform, mode='upper')
    whisper = partialmethod(transform, mode='lower')

tp = TextProcessor()
print(tp.shout('hello'))
print(tp.whisper('HELLO'))
```

partialmethod() is the class-method equivalent of partial(): it creates a new method with some arguments pre-filled, defined directly in the class body, correctly handling the automatic self argument that ordinary partial() would not know how to deal with cleanly on an unbound method.

9.4 wraps()

```
from functools import wraps

def logged(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
```

```

        print(f'Calling {func.__name__}')
        return func(*args, **kwargs)
    return wrapper

@logged
def add(a, b):
    '''Add two numbers.'''
    return a + b

print(add.__name__)
print(add.__doc__)

```

Code Line	Explanation
The problem without @wraps	Without it, <code>add.__name__</code> would print 'wrapper' and <code>add.__doc__</code> would print None, since the decorator replaces <code>add</code> with <code>wrapper</code> entirely, silently losing the original function's identity and documentation.
<code>@wraps(func)</code>	Copies <code>func's __name__, __doc__,</code> and several other metadata attributes onto <code>wrapper</code> , so the decorated function still introspects and behaves like the original for debugging, documentation tools, and tracebacks. Section 20 revisits this in the full context of writing decorators.

9.5 lru_cache()

```

from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(30))

```

Code Line	Explanation
What it does	Automatically caches a function's return values keyed by its arguments; calling <code>fibonacci(30)</code> again later returns the cached result instantly instead of recomputing it, and even the recursive calls within a single computation benefit from previously cached sub-results.
<code>maxsize=128</code>	Limits the cache to the 128 most recently used distinct argument combinations, evicting the least-recently-used entry once that limit is exceeded (an LRU, Least Recently Used, cache); <code>maxsize=None</code> makes the cache unbounded.
Requirement	All arguments must be hashable (so, for example, a list argument would fail); this ties directly back to the immutability discussion in section 13, since hashability generally requires immutability.

9.6 cache()

```
from functools import cache

@cache
def expensive_lookup(key):
    print(f'Computing for {key}')
    return key.upper()
```

cache() (added in Python 3.9) is simply lru_cache(maxsize=None) under a shorter, clearer name, for the common case of an unbounded cache; use it when there is no practical need to limit the cache's size.

9.7 cached_property()

```
from functools import cached_property

class Report:
    def __init__(self, data):
        self.data = data

    @cached_property
    def summary(self):
        print('Computing summary...')
        return {'total': sum(self.data), 'count': len(self.data)}

r = Report([1, 2, 3, 4])
print(r.summary)
print(r.summary)
```

'Computing summary...' prints only once; the first access computes and stores the result directly on the instance, and every subsequent access reuses the stored value without recomputing, combining the convenience of a plain attribute with the laziness of computing it only when actually needed.

9.8 singledispatch()

```
from functools import singledispatch

@singledispatch
def describe(value):
    return f'A value: {value}'

@describe.register
def _(value: int):
    return f'An integer: {value}'

@describe.register
def _(value: list):
    return f'A list with {len(value)} items'

print(describe(42))
print(describe([1, 2, 3]))
print(describe('hello'))
```

Code Line	Explanation
@singledispatch	Turns describe() into a generic function that dispatches to a different implementation based on the type of its first argument, similar in spirit to function overloading in other languages.
@describe.register	Registers a specialized implementation for a specific type, detected from the type annotation on the parameter; calling describe(42) automatically routes to the int-specific version, describe([1,2,3]) to the list-specific version, and anything else (like a string) falls back to the original generic implementation.

9.9 singledispatchmethod()

```

from functools import singledispatchmethod

class Formatter:
    @singledispatchmethod
    def format(self, value):
        return str(value)

    @format.register
    def _(self, value: float):
        return f'{value:.2f}'

f = Formatter()
print(f.format(42))
print(f.format(3.14159))

```

The class-method equivalent of singledispatch(), correctly accounting for self as the first parameter and dispatching based on the type of the next argument instead.

10. operator Module

The operator module provides every one of Python's built-in operators (arithmetic, comparison, and more) as ordinary, callable functions. This matters specifically for functional programming because functions like `map()`, `reduce()`, and `sorted(key=...)` all expect a callable, and operator's functions let common operations be passed directly, without wrapping them in a throwaway lambda.

10.1 Arithmetic Operators as Functions

```
from operator import add, sub, mul, truediv, floordiv, mod, pow
print(add(3, 4))
print(sub(10, 3))
print(mul(6, 7))
print(truediv(7, 2))
print(floordiv(7, 2))
print(mod(7, 2))
print(pow(2, 10))
```

Code Line	Explanation
<code>add, sub, mul</code>	Function equivalents of <code>+</code> , <code>-</code> , <code>*</code> respectively, taking two positional arguments.
<code>truediv, floordiv, mod, pow</code>	Function equivalents of <code>/</code> , <code>//</code> , <code>%</code> , <code>**</code> respectively; each name maps directly to the operator it replaces (recall the arithmetic operator definitions from your Operators notes).

10.2 A Practical Use: With `reduce()`

```
from functools import reduce
from operator import add, mul
numbers = [1, 2, 3, 4]
total = reduce(add, numbers)
product = reduce(mul, numbers)
print(total, product)
```

`reduce(add, numbers)` is clearer and slightly faster than `reduce(lambda acc, x: acc + x, numbers)`, since it avoids both the lambda's overhead and its extra visual noise; this is one of the most common, genuinely idiomatic uses of the operator module.

10.3 Comparison Operators as Functions

```
from operator import eq, ne, lt, le, gt, ge
print(eq(5, 5))
print(lt(3, 7))
print(ge(10, 10))
```

Function equivalents of `==`, `!=`, `<`, `<=`, `>`, `>=` respectively; useful anywhere a comparison needs to be passed as a callable, such as a custom sort or a configurable filtering function that accepts a comparison operator as a parameter.

10.4 itemgetter, attrgetter, methodcaller Revisited

These three (covered fully with examples in sections 8.8-8.10) are also part of the operator module and are, in practice, its most frequently used members in everyday functional-style Python, specifically because they eliminate the need for a lambda in the extremely common 'get this field/attribute/method result' pattern used throughout sorting, mapping, and filtering.

10.5 A Combined Practical Example

```
from operator import itemgetter
from functools import reduce

sales = [
    {'region': 'East', 'amount': 100},
    {'region': 'West', 'amount': 250},
    {'region': 'East', 'amount': 175},
]

total_sales = reduce(lambda acc, s: acc + s['amount'], sales, 0)
sorted_sales = sorted(sales, key=itemgetter('amount'), reverse=True)
print(total_sales)
print(sorted_sales)
```

Combining operator's itemgetter for sorting with a reduce() accumulation demonstrates how these tools compose naturally together in realistic data-processing code, a theme developed fully in the pipeline sections (21-22).

11. Partial Functions

11.1 What Partial Application Means

Partial application is the technique of taking a function that accepts several arguments and producing a new function with some of those arguments already fixed, leaving only the remaining ones to be supplied later. `functools.partial()` is Python's direct, built-in tool for this. It is conceptually related to, but distinct from, currying (transforming a multi-argument function into a chain of single-argument functions), which Python does not build in natively but which `partial()` can approximate.

11.2 Creating Specialized Functions

```
from functools import partial

def greet(greeting, name):
    return f'{greeting}, {name}!'

say_hello = partial(greet, 'Hello')
say_goodbye = partial(greet, 'Goodbye')

print(say_hello('Ada'))
print(say_goodbye('Ada'))
```

Code Line	Explanation
<code>partial(greet, 'Hello')</code>	Creates a new callable that behaves like <code>greet</code> , but with the first positional argument permanently fixed to <code>'Hello'</code> ; calling <code>say_hello('Ada')</code> is exactly equivalent to calling <code>greet('Hello', 'Ada')</code> .
Comparison to section 3.3's closures	This produces essentially the same practical result as the <code>make_validator()</code> closure pattern from section 3.3, but <code>partial()</code> is a more direct, declarative, and slightly more efficient tool specifically for the 'fix some arguments' use case, without needing to write an explicit nested function.

11.3 Passing Fixed Arguments by Keyword

```
from functools import partial

def connect(host, port, timeout=30, retries=3):
    return f'Connecting to {host}:{port} (timeout={timeout}, retries={retries})'

connect_local = partial(connect, host='localhost', port=8080)
print(connect_local())
print(connect_local(timeout=60))
```

Code Line	Explanation
<code>partial(connect, host='localhost', port=8080)</code>	Fixes <code>host</code> and <code>port</code> by keyword, leaving <code>timeout</code> and <code>retries</code> free to be supplied (or left at their defaults) at call time.
<code>connect_local(timeout=60)</code>	Additional keyword arguments passed when calling the partial

Code Line	Explanation
	override or supplement the ones fixed at creation time; here timeout=60 is used instead of the underlying function's own default of 30.

11.4 Partial Functions in Data Processing

```
from functools import partial

def normalize(value, min_val, max_val):
    return (value - min_val) / (max_val - min_val)

normalize_score = partial(normalize, min_val=0, max_val=100)
scores = [45, 78, 92, 60]
normalized = list(map(normalize_score, scores))
print(normalized)
```

Creating a specialized `normalize_score` function with fixed `min_val` and `max_val`, then using it directly with `map()`, is a clean, reusable way to parameterize a general transformation function for a specific dataset, without rewriting the underlying logic or resorting to a lambda that closes over the min/max values manually.

11.5 Partial Functions in APIs and ML Pipelines

```
from functools import partial
import requests

def api_request(base_url, endpoint, api_key, params=None):
    return requests.get(
        f'{base_url}/{endpoint}',
        headers={'Authorization': f'Bearer {api_key}'},
        params=params
    )

call_api = partial(api_request, base_url='https://api.example.com', api_key=API_KEY)
users = call_api('users')
orders = call_api('orders', params={'status': 'active'})
```

Fixing `base_url` and `api_key` once with `partial()` produces a specialized, ready-to-use client function, avoiding repeating that boilerplate at every call site; this pattern scales naturally into ML pipelines too, for example `partial(preprocess, scaler=fitted_scaler)` to bind a fitted preprocessing object into a reusable transformation step, revisited concretely in section 18.

12. Closures

12.1 Nested Functions and Enclosing Scope

```
def outer():
    message = 'Hello from outer'
    def inner():
        print(message)
    return inner

my_func = outer()
my_func()
```

Code Line	Explanation
def inner(): print(message)	inner() is defined inside outer() and references message, a variable from outer()'s local scope, the enclosing scope relative to inner().
return inner (not inner())	Returns the function object itself, not its result; outer() finishes executing and would normally have its local variables (including message) discarded, except for what happens next.
my_func()	Even though outer() has already finished running, calling my_func() still correctly prints 'Hello from outer'; inner() has somehow retained access to message from a scope that technically no longer exists as an active function call. This is exactly what a closure is.

12.2 What a Closure Actually Is

A closure is a function object that remembers values from its enclosing lexical scope, even after that enclosing scope has finished executing. Python achieves this by having the inner function carry a reference to each specific variable it uses from the enclosing scope, packaged as part of the function object itself, rather than the variable simply living on a now-destroyed stack frame.

12.3 Capturing Variables

```
def make_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

c1 = make_counter()
print(c1())
print(c1())
print(c1())

c2 = make_counter()
print(c2())
```

Code Line	Explanation
<code>nonlocal count</code>	Without this, <code>count += 1</code> inside <code>counter()</code> would raise an <code>UnboundLocalError</code> ; Python's default rule is that assigning to a variable inside a function makes it local to that function, which would shadow, rather than modify, the enclosing <code>count</code> . <code>nonlocal</code> explicitly tells Python this <code>count</code> refers to the enclosing scope's variable, not a new local one.
<code>c1()</code> called three times	Prints 1, then 2, then 3; each call to <code>c1()</code> increments and remembers the SAME <code>count</code> variable across calls, because <code>c1</code> is one specific closure with its own captured <code>count</code> .
<code>c2 = make_counter()</code>	Creates a completely separate closure with its own independent <code>count</code> , starting fresh at 0; <code>c2()</code> prints 1, confirming <code>c1</code> and <code>c2</code> do not share state.

12.4 The `nonlocal` Keyword

`nonlocal` is required specifically when an inner function needs to reassign (not merely read) a variable from its enclosing function's scope. Simply reading an enclosing variable, as in the message example in section 12.1, requires no special keyword at all; `nonlocal` is needed only for the counter-style pattern where the closure must modify its captured state across multiple calls.

12.5 The Closure Lifecycle

- 1. The outer function begins executing and creates its local variables.
- 2. The inner function is defined, and Python identifies which enclosing-scope variables it actually references.
- 3. The outer function returns the inner function object; the specific variables the inner function needs are kept alive, bundled with it, rather than being garbage collected along with the rest of the outer function's now-finished stack frame.
- 4. Every time the returned inner function is called, it operates on that same bundled, persistent state, even though the outer function itself is long finished.

12.6 Inspecting Closures

```
def make_multiplier(factor):
    def multiplier(x):
        return x * factor
    return multiplier

double = make_multiplier(2)
print(double.__closure__)
print(double.__closure__[0].cell_contents)
```

Code Line	Explanation
<code>double.__closure__</code>	Every closure exposes a <code>__closure__</code> attribute: a tuple of 'cell' objects, one for each captured variable, confirming directly that the function object really is carrying this extra state around with it.
<code>cell_contents</code>	Each cell object's <code>.cell_contents</code> attribute reveals the actual captured value, here 2, the factor <code>double</code> remembers.

12.7 Practical Examples

```
def make_validator(rule_name, condition):
    def validate(value):
        if not condition(value):
            raise ValueError(f'{rule_name} failed for value: {value}')
        return True
    return validate

is_positive = make_validator('positive check', lambda x: x > 0)
is_positive(5)
is_positive(-3)
```

A closure factory that produces a family of reusable, self-describing validation functions, each remembering its own `rule_name` and `condition`; this pattern connects directly to the custom exception validation patterns you may have seen in your Exception Handling notes, expressed here in a purely functional style instead of inside a class.

12.8 Closures vs Global Variables

```
# Global-state version: fragile, shared, hard to test in isolation
count = 0
def increment_global():
    global count
    count += 1
    return count

# Closure version: encapsulated, independent, testable
def make_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter
```

The global version has exactly one count shared across the entire program, uncontrolled by any caller, at risk of being modified unexpectedly from anywhere else in the codebase, and hard to reset or isolate for a test. The closure version can be instantiated independently as many times as needed (section 12.3's `c1` and `c2`), each with fully isolated, private state, a direct practical benefit of the encapsulation closures provide over module-level global variables.

12.9 Closures vs Classes

Code Line	Explanation
A closure	A lightweight way to bundle a small amount of private state with a single behavior (one function); simple, concise, and appropriate when only one operation is needed on the captured state.
A class	More appropriate when multiple related behaviors need to share and operate on the same state (multiple methods), or when the state itself needs a clear, named, documented structure; your OOP notes cover this in full.

The `make_counter()` closure and an equivalent `Counter` class with an `increment()` method and an internal `self.count` attribute accomplish very similar things for the single-method case; closures tend to be preferred for small, single-purpose stateful functions, while classes are preferred once more than one related operation on the same state is needed.

12.10 Connection to Decorators

Every decorator that adds behavior around a function (like the `@wraps` and `@logged` examples in sections 9.4 and 3.4) relies on a closure: the wrapper function captures and remembers the original func from its enclosing scope, exactly following the closure mechanics explained throughout this section. Section 20 builds this connection out completely, showing decorators as a direct, natural application of closures and higher-order functions together.

13. Immutability and Side Effects

13.1 Mutable vs Immutable Objects: A Functional Lens

Your Data Types notes already cover which built-in types are mutable (list, dict, set) versus immutable (int, float, str, tuple, frozenset) in detail. This section focuses specifically on why this distinction matters so much to functional programming, and the practical strategies for working with it deliberately.

13.2 Why Functional Programming Prefers Immutability

```
def add_discount(cart, item):
    cart.append(item) # mutates the caller's original list
    return cart

original_cart = ['apple', 'bread']
new_cart = add_discount(original_cart, 'milk')
print(original_cart)
```

Code Line	Explanation
The problem	original_cart is silently mutated by add_discount(), even though the function's return value suggests it produces a new cart; printing original_cart afterward reveals it now also contains 'milk', which may be a complete surprise to whatever other code still holds a reference to original_cart and expected it to remain unchanged.
Why this matters functionally	Functional style avoids exactly this class of bug by preferring to build and return new data rather than mutating shared input in place, making a function's effects fully visible in its return value, with nothing hidden.

```
def add_discount(cart, item):
    return cart + [item] # returns a new list, leaves the original untouched

original_cart = ['apple', 'bread']
new_cart = add_discount(original_cart, 'milk')
print(original_cart)
print(new_cart)
```

cart + [item] creates a brand new list, entirely independent of the original; original_cart remains exactly as it was, and the caller can decide, explicitly, whether to keep using the old cart or switch to new_cart.

13.3 Global-State Problems

```
cache = {}

def get_user(user_id):
    if user_id not in cache:
        cache[user_id] = fetch_from_database(user_id)
    return cache[user_id]
```

get_user() silently depends on and mutates a module-level global dictionary; calling it twice with the same user_id can return different results depending on unrelated code elsewhere that might have already populated or

cleared cache, making the function's behavior depend on hidden, hard-to-trace external state rather than purely on its own argument, exactly the opposite of the referential transparency introduced in section 1.6.

13.4 Pure Transformations

```
def apply_discount(price, percent):
    return price * (1 - percent / 100)

original_prices = [100, 200, 50]
discounted_prices = [apply_discount(p, 10) for p in original_prices]
print(original_prices)
print(discounted_prices)
```

`apply_discount()` is a pure transformation: it takes a price and returns a new, discounted price, with no dependency on or effect on anything outside its own arguments and return value. Building `discounted_prices` as a brand new list (via a comprehension, section 15) rather than modifying `original_prices` in place keeps both pieces of data available, predictable, and independently trustworthy.

13.5 Avoiding Unnecessary Mutation

```
# Mutating style
def normalize_names(names):
    for i in range(len(names)):
        names[i] = names[i].strip().title()
    return names

# Functional, non-mutating style
def normalize_names(names):
    return [name.strip().title() for name in names]
```

Both versions produce a list of cleaned names, but the first silently overwrites the caller's original list in place, while the second leaves the input entirely untouched and returns a new list; the second version is safer specifically because its effect is fully contained in its return value, with nothing to discover by reading beyond the function's signature.

13.6 Functional Approaches to Data Transformation

- Prefer returning new values (via comprehensions, `map()`, or straightforward expressions) over mutating arguments in place.
- Treat function parameters as read-only inputs by convention, even where Python does not enforce this; never reassign or mutate a mutable parameter unless that mutation is explicitly the function's documented, intended purpose.
- When a genuine running total or accumulated state is required, isolate it clearly, inside a single function's local scope, a closure (section 12), or an explicit `reduce()` accumulator (section 7), rather than scattering mutations across module-level global variables.

13.7 Python's Limitations Regarding True Immutability

```
point = (1, [2, 3])
```

```
point[1].append(4)
print(point)
```

Code Line	Explanation
The surprise	point is a tuple, which is immutable, meaning you cannot reassign point[0] or point[1] to a different object. But point[1] itself is a mutable list, and its contents CAN still be changed; point becomes (1, [2, 3, 4]) despite point itself being a tuple.
The lesson	Immutability in Python is shallow by default: an immutable container only guarantees its own direct references cannot be reassigned, not that everything nested inside it is also immutable. Genuinely deep immutability requires deliberately using immutable types (tuples instead of lists, frozenset instead of set) at every level of a nested structure, or explicit, disciplined copying.

This is an honest, important limitation: Python was never designed to enforce immutability the way some purely functional languages do, and writing genuinely immutable-style code in Python requires ongoing discipline rather than relying on the language to prevent mutation for you.

14. Pure Functions

14.1 Definition (Full Treatment)

A pure function satisfies exactly two properties, both already introduced briefly in section 1.5: given the same input, it always produces the same output (deterministic), and it causes no observable side effects, it does not modify any state outside its own local scope, and it does not depend on any state beyond its own parameters.

14.2 Characteristics of Pure Functions

- Deterministic: identical arguments always yield an identical result, every single time, with no exceptions.
- No side effects: no mutation of arguments, no writes to global variables, no I/O of any kind (printing, file writes, network calls).
- No dependency on external, mutable state: the function's behavior is fully determined by its explicit parameters alone.
- Referentially transparent (section 1.6): a call to the function can always be safely replaced by its return value with no change in program behavior.

14.3 Examples of Pure Functions

```
def add(a, b):  
    return a + b  
  
def calculate_bmi(weight_kg, height_m):  
    return weight_kg / (height_m ** 2)  
  
def format_currency(amount, symbol='$'):  
    return f'{symbol}{amount:,.2f}'
```

Each of these depends only on its own explicit arguments, produces a return value with no side effect, and would always produce the exact same output for the exact same input, no matter how many times or in what order they are called.

14.4 Impure Functions and Their Common Causes

14.4.1 Randomness

```
import random  
def roll_die():  
    return random.randint(1, 6)
```

`roll_die()` takes no arguments at all, yet returns a different result across calls; it is impure by definition, since it violates determinism directly.

14.4.2 File Operations

```
def load_config():  
    with open('config.json') as f:  
        return json.load(f)
```

Reading from a file is an impure operation: the result depends on the file's current contents, an external, mutable piece of state outside the function's own arguments, and the function also performs I/O, a side effect in its own right.

14.4.3 Database Operations and API Calls

```
def get_current_price(product_id):
    return database.query(f'SELECT price FROM products WHERE id = {product_id}')
```

Depends on the current, changing state of an external database; the same `product_id` can legitimately return a different price at different times, making this function impure by nature, not by poor design, since querying live external data is precisely its purpose.

14.4.4 Global Variables

```
tax_rate = 0.08
def calculate_total(price):
    return price * (1 + tax_rate)
```

Depends on a global variable rather than an explicit parameter; the same price argument can produce different results if `tax_rate` is changed elsewhere in the program between calls, making this function's behavior dependent on hidden context rather than fully determined by its visible arguments.

14.5 Designing More Predictable Functions

```
# Impure version
tax_rate = 0.08
def calculate_total(price):
    return price * (1 + tax_rate)

# Pure version: tax_rate becomes an explicit parameter
def calculate_total(price, tax_rate):
    return price * (1 + tax_rate)
```

The fix here is simple and broadly applicable: turn implicit dependencies (global variables, external state) into explicit parameters wherever practical. This does not eliminate the need for that value to come from somewhere (a configuration file, a database), but it isolates that impurity to one clearly identified boundary (wherever `tax_rate` is first read), keeping `calculate_total()` itself pure, testable, and reliably predictable.

14.6 Benefits for Testing and Debugging

```
def test_calculate_total():
    assert calculate_total(100, 0.08) == 108.0
    assert calculate_total(0, 0.08) == 0.0
    assert calculate_total(100, 0) == 100.0
```

Code Line	Explanation
Why this test is simple and reliable	Every assertion is completely self-contained: no database to connect to, no global state to set up beforehand or reset afterward, no mocking required, and no risk of test order affecting the outcome. The pure <code>calculate_total()</code> can be tested exhaustively with nothing more than direct input/output pairs.

Code Line	Explanation
Debugging benefit	When a pure function produces a wrong result, the bug must be inside that function's own logic, since there is no hidden external state that could have caused it; this dramatically narrows the search space compared to debugging an impure function that might be failing due to unexpected global state, timing, or external system behavior. Section 26 expands this connection between purity and testing into a complete, dedicated treatment.

15. Comprehensions as Functional-Style Python

15.1 Why Comprehensions Belong in This Handbook

Comprehensions are not, strictly speaking, one of the 'named' functional programming tools like `map()`, `filter()`, or `reduce()`, but they express exactly the same underlying idea, declaratively describing a new collection as a transformation of an existing one, in syntax that is often considered more Pythonic and more readable than the equivalent chained function calls. This section assumes basic familiarity with comprehension syntax from your fundamentals notes and focuses specifically on their role as a functional-programming idiom.

15.2 List Comprehensions as `map()`/`filter()` Replacements

```
numbers = [1, 2, 3, 4, 5, 6]

# map() equivalent
squares = [n ** 2 for n in numbers]

# filter() equivalent
evens = [n for n in numbers if n % 2 == 0]

# map() + filter() combined equivalent
even_squares = [n ** 2 for n in numbers if n % 2 == 0]
```

The third example demonstrates something `map()` and `filter()` cannot do cleanly on their own without composing two separate calls: transforming and filtering in a single, readable expression, a major reason comprehensions are often preferred in idiomatic Python over chaining `list(map(..., filter(..., ...)))`.

15.3 Set Comprehensions

```
words = ['apple', 'Banana', 'apple', 'CHERRY', 'banana']
unique_lower = {w.lower() for w in words}
print(unique_lower)
```

Produces a set of transformed, de-duplicated values in one expression; equivalent to `set(map(str.lower, words))`, but generally considered more directly readable.

15.4 Dictionary Comprehensions

```
names = ['Ada', 'Grace', 'Alan']
name_lengths = {name: len(name) for name in names}
print(name_lengths)

prices = {'apple': 1.50, 'banana': 0.75, 'cherry': 4.00}
discounted = {item: round(price * 0.9, 2) for item, price in prices.items()}
print(discounted)
```

The second example is a direct, functional-style transformation of an entire dictionary's values while preserving its keys, a pattern that would otherwise require `dict(map(...))` with a lambda producing (key, value) tuples, noticeably less direct than the comprehension form.

15.5 Generator Expressions

```
numbers = [1, 2, 3, 4, 5]
squares_gen = (n ** 2 for n in numbers)
print(squares_gen)
print(sum(squares_gen))
```

Code Line	Explanation
(n ** 2 for n in numbers)	Parentheses instead of square brackets create a generator expression instead of a list; like map(), this is lazy, values are computed one at a time as they are requested, not all upfront.
print(squares_gen)	Prints a generator object description, not the values themselves, confirming nothing has been computed yet.
sum(squares_gen)	Consuming the generator (here, by summing it) triggers the actual computation, one value at a time; a generator expression can only be consumed once, exactly like a map() object.

Section 23 covers lazy evaluation, including generator expressions, in complete formal depth, including precisely why this laziness matters for memory efficiency with large datasets.

15.6 Conditional Comprehensions

```
numbers = [-3, -1, 0, 2, 5]
labels = ['positive' if n > 0 else 'non-positive' for n in numbers]
print(labels)
```

Code Line	Explanation
Conditional EXPRESSION (before the for)	'positive' if n > 0 else 'non-positive' transforms every element, choosing between two output values based on a condition; every input element produces exactly one output element.
Conditional FILTER (an if after the for)	[n for n in numbers if n > 0] instead removes elements entirely; some input elements may produce no corresponding output element at all. These two uses of if inside a comprehension serve genuinely different purposes and are not interchangeable.

15.7 Nested Comprehensions

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [value for row in matrix for value in row]
print(flattened)

transposed = [[row[i] for row in matrix] for i in range(3)]
print(transposed)
```

Code Line	Explanation
flattened	Two for clauses in one comprehension flatten a nested list into a single flat list; read left to right, the outer for row in matrix runs first, and for each row, the inner for value in row runs.
transposed	A comprehension nested inside another comprehension builds a genuinely new nested structure (here, the matrix's transpose); this is

Code Line	Explanation
	meaningfully harder to read at a glance than the flattening example, illustrating the readability boundary discussed next.

15.8 Comprehensions vs map() and filter()

Code Line	Explanation
Favor a comprehension	When combining a transformation and a filter together, when the transformation is a short inline expression, or when building a set/dict rather than a list (map()/filter() always require an extra wrapping call to build anything other than their own lazy iterator type).
Favor map()/filter()	When the transformation function already exists, is named, and is reused elsewhere, avoiding the need to re-express its logic inline as a comprehension's expression.

15.9 Readability and Performance

Comprehensions are generally at least as fast as, and often faster than, an equivalent explicit loop with `.append()`, because the looping mechanism is implemented in optimized C code internally. Compared to `map()/filter()`, performance differences are typically negligible for anything but extremely large datasets or extremely tight inner loops; readability, not raw speed, should be the primary factor in choosing between them for the overwhelming majority of real code.

15.10 When Comprehensions Become Too Complicated

```
# Hard to read: too much logic packed into one line
result = [
    {'id': r['id'], 'total': r['price'] * r['qty'] * (1 - r.get('discount', 0))}
    for r in records if r['price'] > 0 and r['qty'] > 0 and r.get('active', True)
]
```

Once a comprehension needs multiple conditions, a nested transformation, and a fallback default all packed into one dense expression, it starts working against its own purpose, which is to communicate intent clearly and concisely. At this point, an ordinary for loop with clearly named intermediate variables, or a small, well-named helper function combined with a simpler comprehension, communicates the same logic far more clearly. There is no strict line, but if a comprehension no longer fits comfortably on one or two lines and requires real effort to parse, it has likely crossed it; this exact judgment call is revisited in section 28.3 and section 29.

16. Functional Programming with Strings, Lists, Tuples, Sets, and Dictionaries

This section applies the tools from sections 4 through 15 directly and practically across each of Python's core built-in collection types, without re-teaching their basic mechanics, which your Data Types notes already cover.

16.1 Strings

```
sentence = 'the quick brown fox jumps'
word_lengths = list(map(len, sentence.split()))
long_words = list(filter(lambda w: len(w) > 4, sentence.split()))
capitalized = ' '.join(map(str.capitalize, sentence.split()))
print(word_lengths, long_words, capitalized)
```

Splitting a string into words and then applying `map()`/`filter()` over that list of words is one of the most common functional-style text-processing patterns, foundational to the NLP examples developed fully in section 19.

16.2 Lists

```
orders = [120, 45, 300, 15, 80]
big_orders = list(filter(lambda x: x > 100, orders))
with_tax = list(map(lambda x: round(x * 1.08, 2), orders))
from functools import reduce
total = reduce(lambda acc, x: acc + x, orders, 0)
```

16.3 Tuples

```
coordinates = [(1, 2), (3, 4), (-1, 5)]
distances = list(map(lambda p: (p[0] ** 2 + p[1] ** 2) ** 0.5, coordinates))
in_first_quadrant = list(filter(lambda p: p[0] > 0 and p[1] > 0, coordinates))
```

Tuples pair naturally with functional transformations specifically because their immutability (per section 13) guarantees the original coordinate data can never be accidentally altered by a transformation step, an assurance a list of mutable objects could not offer as strongly.

16.4 Sets

```
tags_a = {'python', 'data', 'ml'}
tags_b = {'python', 'web', 'api'}
shared = tags_a & tags_b
combined_upper = set(map(str.upper, tags_a | tags_b))
print(shared, combined_upper)
```

Set operations (`&` for intersection, `|` for union, covered fully in your Operators notes) are themselves inherently functional: each one takes two sets and produces a brand new set, with neither original set mutated, a natural fit for the functional style.

16.5 Dictionaries

```
inventory = {'apples': 50, 'bananas': 0, 'cherries': 30}
```

```
in_stock = dict(filter(lambda kv: kv[1] > 0, inventory.items()))
doubled = {k: v * 2 for k, v in inventory.items()}
total_stock = sum(inventory.values())
print(in_stock, doubled, total_stock)
```

Since dictionaries have no direct `map()`/`filter()` equivalent of their own, working through `.items()` (as shown throughout sections 5, 6, and here) is the standard, idiomatic bridge that lets every functional tool in this handbook apply cleanly to dictionary data as well.

17. Functional Programming for Data Analytics

17.1 Cleaning Data

```
raw_records = [
    {'name': ' Ada ', 'age': '34'},
    {'name': 'Alan', 'age': None},
    {'name': '', 'age': '29'},
]

def clean_record(r):
    return {'name': r['name'].strip(), 'age': r['age']}

cleaned = list(map(clean_record, raw_records))
valid = list(filter(lambda r: r['name'] and r['age'] is not None, cleaned))
print(valid)
```

A two-stage pipeline: `map()` normalizes every record's shape (trimming whitespace), then `filter()` removes records that remain invalid after cleaning; separating these two concerns into distinct, composable steps is a core functional data-cleaning pattern, expanded into a complete named pipeline in section 22.

17.2 Transforming Columns

```
sales_records = [
    {'product': 'Widget', 'price': 9.99, 'qty': 3},
    {'product': 'Gadget', 'price': 19.99, 'qty': 1},
]

with_totals = list(map(
    lambda r: {**r, 'total': round(r['price'] * r['qty'], 2)},
    sales_records
))
print(with_totals)
```

Deriving a new column (`total`) from existing ones without mutating the original records, using the `{**r, ...}` pattern first introduced in section 5.8, keeps the transformation traceable and side-effect free.

17.3 Filtering Records

```
customers = [
    {'name': 'Ada', 'region': 'East', 'ltv': 1200},
    {'name': 'Alan', 'region': 'West', 'ltv': 300},
    {'name': 'Grace', 'region': 'East', 'ltv': 950},
]

high_value_east = list(filter(
    lambda c: c['region'] == 'East' and c['ltv'] > 500,
    customers
))
```

17.4 Aggregating Values

```
from functools import reduce

sales_by_region = reduce(
    lambda acc, r: {**acc, r['region']: acc.get(r['region'], 0) + r['ltv']},
    customers,
    {}
)
print(sales_by_region)
```

This directly reuses the record-combining `reduce()` pattern from section 7.8, here computing a region-level aggregate from raw customer records, a functional equivalent of a `groupby-and-sum` operation.

17.5 Sorting Data

```
from operator import itemgetter
ranked_customers = sorted(customers, key=itemgetter('ltv'), reverse=True)
```

17.6 Creating Derived Values

```
transactions = [
    {'amount': 250, 'type': 'credit'},
    {'amount': -80, 'type': 'debit'},
]

signed = list(map(
    lambda t: t['amount'] if t['type'] == 'credit' else -abs(t['amount']),
    transactions
))
print(signed)
```

17.7 Processing Transaction, Customer, Sales, and Financial Records

```
financial_records = [
    {'account': 'A1', 'balance': 1500.0, 'currency': 'USD'},
    {'account': 'A2', 'balance': -200.0, 'currency': 'USD'},
    {'account': 'A3', 'balance': 800.0, 'currency': 'EUR'},
]

usd_only = list(filter(lambda a: a['currency'] == 'USD', financial_records))
overdrawn = list(filter(lambda a: a['balance'] < 0, financial_records))
total_usd = sum(map(lambda a: a['balance'], usd_only))
print(usd_only, overdrawn, total_usd)
```

17.8 Working With Pandas: Where Pure Python FP Fits

```
import pandas as pd

df = pd.DataFrame(sales_records)
```

```
df['total'] = df['price'] * df['qty']

df['category'] = df['product'].apply(lambda p: 'Premium' if p == 'Gadget' else
'Standard')
```

Code Line	Explanation
df['total'] = df['price'] * df['qty']	For simple element-wise arithmetic across whole columns, pandas' own vectorized operations (covered fully in your Pandas notes) are always faster and more idiomatic than map()/lambda applied row by row; this is NOT a place to reach for pure Python functional tools.
df['category'] = df['product'].apply(lambda p: ...)	pandas' own .apply() accepts a plain Python function or lambda directly, which is where functional-style thinking, and specifically well-designed pure functions, still applies naturally and usefully inside a pandas workflow, for logic too complex or conditional for a simple vectorized expression.
Important Distinction: Pure Python map()/filter()/reduce() are not a replacement for pandas' own vectorized operations when working with a DataFrame; pandas is almost always faster and more idiomatic for column-wide numeric operations. Functional programming techniques remain valuable specifically for the logic INSIDE a .apply() call, for preparing raw records before they become a DataFrame, and for any data still represented as plain Python lists/dicts rather than already inside pandas.	

18. Functional Programming for Data Science and Machine Learning

18.1 Feature Engineering

```
records = [
    {'height_cm': 170, 'weight_kg': 65},
    {'height_cm': 182, 'weight_kg': 90},
]

def add_bmi(record):
    height_m = record['height_cm'] / 100
    bmi = record['weight_kg'] / (height_m ** 2)
    return {**record, 'bmi': round(bmi, 2)}

featured = list(map(add_bmi, records))
print(featured)
```

A pure, named `add_bmi()` feature-engineering function used with `map()` over a dataset is directly reusable, directly testable in isolation (per section 26), and does not mutate the original records, all properties that matter significantly once feature engineering logic grows beyond a single toy example into a real, evolving pipeline.

18.2 Data Preprocessing

```
raw_values = [23, None, 45, -1, 67, None, 12]

def is_valid(x):
    return x is not None and x >= 0

cleaned = list(filter(is_valid, raw_values))
mean = sum(cleaned) / len(cleaned)
imputed = list(map(lambda x: x if is_valid(x) else mean, raw_values))
print(cleaned, mean, imputed)
```

18.3 Transformation Pipelines

```
def scale(values, min_v, max_v):
    return [(v - min_v) / (max_v - min_v) for v in values]

def clip(values, low, high):
    return [max(low, min(high, v)) for v in values]

raw = [5, 50, 95, 120, -10]
scaled = scale(raw, min_v=0, max_v=100)
clipped = clip(scaled, low=0, high=1)
```

Each step (scale, clip) is a small, pure, independently testable transformation function; chaining them together, as shown here manually and more formally in sections 21-22, is the essence of a functional preprocessing pipeline.

18.4 Dataset Filtering

```
dataset = [
    {'features': [1.2, 3.4], 'label': 1},
    {'features': [0.0, 0.0], 'label': 0},
    {'features': [None, 2.1], 'label': 1},
]

valid_dataset = list(filter(
    lambda row: all(f is not None for f in row['features']),
    dataset
))
```

`all(f is not None for f in row['features'])` uses a generator expression (section 15.5) directly inside `all()`, a common, memory-efficient functional idiom for validating every element of a nested structure without building an intermediate list.

18.5 Batch Processing

```
def batch(iterable, size):
    items = list(iterable)
    return [items[i:i + size] for i in range(0, len(items), size)]

data = list(range(10))
batches = batch(data, 3)
print(batches)
```

A pure batching function used to split a large dataset into fixed-size chunks for batch inference or batch training, a pattern revisited with `itertools`' more memory-efficient `islice()` approach in section 24.

18.6 Prediction Transformations

```
raw_predictions = [0.12, 0.87, 0.45, 0.93, 0.51]
labels = list(map(lambda p: 1 if p >= 0.5 else 0, raw_predictions))
confidences = list(map(lambda p: max(p, 1 - p), raw_predictions))
print(labels, confidences)
```

18.7 Evaluation Workflows

```
y_true = [1, 0, 1, 1, 0]
y_pred = [1, 0, 0, 1, 1]

correct = list(filter(lambda pair: pair[0] == pair[1], zip(y_true, y_pred)))
accuracy = len(correct) / len(y_true)
print(accuracy)
```

`zip(y_true, y_pred)` pairs up corresponding true and predicted labels; filtering those pairs down to only the matching ones, then comparing the filtered count to the total, computes accuracy in a fully functional, no-loop style.

18.8 ML Preprocessing Functions and Pipeline Design

```
def build_preprocessing_pipeline(steps):
    def process(data):
        for step in steps:
            data = step(data)
        return data
    return process

preprocess = build_preprocessing_pipeline([
    lambda rows: [r for r in rows if r.get('value') is not None],
    lambda rows: [{**r, 'value': round(r['value'], 2)} for r in rows],
])

result = preprocess([{'value': 3.14159}, {'value': None}, {'value': 2.71828}])
print(result)
```

`build_preprocessing_pipeline()` is a higher-order function (section 3) that returns a new function composed from a list of transformation steps; each step is itself a pure function operating on the whole dataset. This is a direct, practical preview of function composition and pipelines, formalized completely in sections 21 and 22.

19. Functional Programming in AI Engineering

19.1 Data Preprocessing Pipelines

```
def load_and_clean(records):
    return list(filter(
        lambda r: r.get('text') and len(r['text'].strip()) > 0,
        records
    ))
```

19.2 NLP Text Transformations

```
import re

def normalize_text(text):
    text = text.lower()
    text = re.sub(r'^a-z0-9\s', '', text)
    return text.strip()

documents = ['Hello, World!', 'Python & AI...', 'Data-Science 2026']
normalized = list(map(normalize_text, documents))
print(normalized)
```

A pure, single-responsibility `normalize_text()` function applied with `map()` across a whole corpus of documents; keeping each transformation step (lowercasing, stripping punctuation, trimming) as one composable function makes the overall text-cleaning logic easy to test, reorder, or extend, following the same design principles established in section 14.5.

19.3 Token Processing

```
STOPWORDS = {'the', 'a', 'an', 'and', 'or', 'is'}

def tokenize(text):
    return text.split()

def remove_stopwords(tokens):
    return list(filter(lambda t: t not in STOPWORDS, tokens))

text = 'the quick brown fox is jumping'
tokens = remove_stopwords(tokenize(text))
print(tokens)
```

19.4 API Response Transformations

```
api_response = {
    'results': [
        {'id': 1, 'score': 0.92, 'label': 'positive'},
        {'id': 2, 'score': 0.31, 'label': 'negative'},
    ]
}
```

```

}

high_confidence = list(filter(
    lambda r: r['score'] > 0.8,
    api_response['results']
))
labels_only = list(map(lambda r: r['label'], api_response['results']))

```

19.5 LLM Output Processing

```

raw_completions = [
    ' Answer: 42 ',
    'Answer: Paris\n',
    '',
]

def extract_answer(completion):
    cleaned = completion.strip()
    return cleaned.replace('Answer:', '').strip() if cleaned else None

answers = list(filter(None, map(extract_answer, raw_completions)))
print(answers)

```

Code Line	Explanation
<code>map(extract_answer, raw_completions)</code>	Applies the same cleaning/extraction logic to every raw LLM output string.
<code>filter(None, ...)</code>	Reuses the truthy-filtering idiom from section 6.8 to drop any completion that produced no usable answer (an empty string, which is falsy), in one combined, readable pipeline.

19.6 Prompt-Processing Pipelines

```

def build_prompt(template, **kwargs):
    return template.format(**kwargs)

from functools import partial
summarize_prompt = partial(
    build_prompt,
    template='Summarize the following text in {n} sentences:\n\n{text}'
)

prompt = summarize_prompt(n=2, text='Long article content here...')
print(prompt)

```

Using `partial()` (section 11) to fix a specific prompt template creates a specialized, reusable prompt-builder function, a natural, practical application of partial application directly within an LLM-based application's prompt engineering layer.

19.7 Model Prediction Pipelines and Batch Inference

```

def batch(iterable, size):

```

```

    items = list(iterable)
    return [items[i:i + size] for i in range(0, len(items), size)]

def run_batch_inference(model, texts, batch_size=32):
    results = []
    for text_batch in batch(texts, batch_size):
        results.extend(model.predict(text_batch))
    return results

```

Reuses the batching function from section 18.5 directly in an inference context, a realistic pattern for running a model over a large volume of inputs without exceeding memory or API request-size limits.

19.8 ETL Pipelines

```

def extract(source):
    return source['records']

def transform(records):
    return list(map(lambda r: {'**r', 'processed': True}, records))

def load(records, destination):
    destination.extend(records)
    return destination

warehouse = []
raw_source = {'records': [{'id': 1}, {'id': 2}]}
load(transform(extract(raw_source)), warehouse)
print(warehouse)

```

The classic Extract-Transform-Load pattern maps directly onto three composable functions; section 22 builds this exact pattern out into a complete, formal, named pipeline utility.

20. Functional Programming + Decorators

20.1 Building Up From First Principles

This section deliberately reconstructs decorators step by step from the functional concepts already covered, first-class functions (section 2), higher-order functions (section 3), and closures (section 12), so that `@decorator` syntax is understood as a direct consequence of those ideas, not as a separate, unrelated piece of magic syntax.

20.2 Step 1: A Function That Takes a Function

```
def shout(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result.upper()
    return wrapper

def greet(name):
    return f'hello, {name}'

loud_greet = shout(greet)
print(loud_greet('ada'))
```

Code Line	Explanation
<code>shout(func)</code>	A higher-order function (section 3): it accepts <code>greet</code> as an argument and returns a brand new function, <code>wrapper</code> , that calls <code>greet</code> internally and then transforms its result. <code>wrapper</code> is a closure (section 12), since it captures and remembers <code>func</code> from its enclosing scope.
<code>loud_greet = shout(greet)</code>	Manually reassigns <code>greet</code> 's behavior to the enhanced version; <code>greet</code> itself is untouched, and <code>loud_greet</code> is the new, decorated function.

20.3 Step 2: Replacing the Original Name

```
def greet(name):
    return f'hello, {name}'

greet = shout(greet)
print(greet('ada'))
```

Reassigning `greet = shout(greet)` replaces the original `greet` with its decorated version under the same name; every future call to `greet(...)` now goes through `wrapper`. This manual pattern, define a function, then immediately reassign its name to a wrapped version of itself, is exactly what `@decorator` syntax automates.

20.4 Step 3: The `@decorator` Syntax

```
def shout(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result.upper()
    return wrapper
```

```
@shout
def greet(name):
    return f'hello, {name}'

print(greet('ada'))
```

@shout placed directly above def greet(name): is exactly equivalent to writing def greet(name): ... followed by greet = shout(greet), as done manually in section 20.3. This is the complete, literal definition of what a decorator is: nothing more than syntax sugar for 'define this function, then immediately pass it through a higher-order function and keep the result under the same name'.

20.5 Step 4: Preserving Metadata With @wraps

```
from functools import wraps

def shout(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs).upper()
    return wrapper

@shout
def greet(name):
    '''Return a friendly greeting.'''
    return f'hello, {name}'

print(greet.__name__, '-', greet.__doc__)
```

Revisits functools.wraps() (fully explained in section 9.4) now in its complete, natural context: correctly written decorators should always preserve the original function's __name__ and __doc__ so that tracebacks, documentation tools, and introspection continue to reflect the true, original function rather than a generic 'wrapper'.

20.6 Decorators With Arguments

```
from functools import wraps

def repeat(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            results = []
            for _ in range(times):
                results.append(func(*args, **kwargs))
            return results
        return wrapper
    return decorator

@repeat(times=3)
def roll():
    import random
```

```

    return random.randint(1, 6)

print(roll())

```

Code Line	Explanation
Three levels of nested functions	repeat(times) is a function that returns decorator, which is itself a decorator (accepts func, returns wrapper); this extra outer layer is what allows repeat(times=3) to accept its own configuration argument before it ever receives the function being decorated.
@repeat(times=3)	First calls repeat(times=3), which returns the actual decorator function; that returned decorator is then applied to roll, exactly following the section 20.4 pattern, just with one additional layer of function-returning-function.

20.7 Function Composition Inside Decorators

A decorator is, in essence, a special case of function composition (formally covered next, in section 21): shout(greet) composes an uppercasing transformation with greet's own output. Understanding decorators through this lens, as composing a wrapping behavior around an existing function, unifies the seemingly separate topics of decorators and function composition into one coherent functional idea.

21. Function Composition

21.1 What Composition Means

Function composition is the act of combining two or more functions so that the output of one becomes the input of the next, producing a single new function representing the whole chain. In mathematical notation, composing f and g (applying g first, then f) is written $(f \circ g)(x) = f(g(x))$. Python has no built-in composition operator, but composition is straightforward to express directly, and to build reusable utilities for.

21.2 Manual Composition

```
def double(x):
    return x * 2

def increment(x):
    return x + 1

result = increment(double(5))
print(result)
```

`increment(double(5))` is composition written out manually and explicitly: `double` runs first, producing 10, and `increment` then runs on that result, producing 11. This is composition in its simplest, most direct form, no special syntax or utility required.

21.3 Creating a Reusable Composition Utility

```
def compose(f, g):
    def composed(x):
        return f(g(x))
    return composed

increment_then_double = compose(double, increment)
print(increment_then_double(5))
```

Code Line	Explanation
<code>compose(f, g)</code>	A higher-order function that returns a new function, composed, applying g first and then f to its result; this directly reuses the closure mechanics from section 12, since <code>composed</code> captures and remembers both f and g .
<code>compose(double, increment)</code>	Reads right to left, mathematically: g (<code>increment</code>) runs first, then f (<code>double</code>); <code>increment_then_double(5)</code> computes <code>double(increment(5)) = double(6) = 12</code> .

21.4 Composing Multiple Functions

```
from functools import reduce

def compose_all(*functions):
    def composed(x):
```

```

        return reduce(lambda acc, f: f(acc), reversed(functions), x)
    return composed

pipeline = compose_all(str.strip, str.lower, str.title)
print(pipeline(' hello WORLD '))

```

Code Line	Explanation
<code>compose_all(*functions)</code>	Accepts any number of functions and returns one combined function; internally, it uses <code>reduce()</code> (section 7) to fold the initial value <code>x</code> through every function in turn, a genuinely practical application of <code>reduce()</code> beyond simple numeric accumulation, directly connecting sections 7 and 21.
<code>reversed(functions)</code>	Ensures the functions are applied right to left, matching traditional mathematical composition order, so <code>compose_all(f, g, h)(x)</code> computes <code>f(g(h(x)))</code> .

21.5 Left-to-Right vs Right-to-Left Composition

```

def pipe(*functions):
    def piped(x):
        return reduce(lambda acc, f: f(acc), functions, x)
    return piped

pipeline = pipe(str.strip, str.lower, str.title)
print(pipeline(' hello WORLD '))

```

Code Line	Explanation
<code>compose_all</code> (section 21.4)	Applies functions right to left, mirroring mathematical notation ($f \circ g$) ($x = f(g(x))$); reads naturally as 'f of g of x'.
<code>pipe</code> (this section)	Applies functions left to right, in the exact order they are listed, which many developers find more intuitive to read, since it matches the visual reading order and the everyday meaning of a 'pipeline' or sequence of steps.

A Note on Convention: Neither direction is objectively correct; `compose_all` (right to left) matches mathematical convention and other functional languages, while `pipe` (left to right) often matches how people naturally think about a sequence of processing steps, especially in data pipelines. Choose whichever convention your team documents clearly and applies consistently.

21.6 Practical Data-Processing Example

```

def remove_punctuation(text):
    import re
    return re.sub(r'[^w\s]', '', text)

def collapse_whitespace(text):
    return ' '.join(text.split())

clean_text = pipe(str.lower, remove_punctuation, collapse_whitespace, str.strip)
print(clean_text(' Hello, World!! Data--Science... '))

```

A realistic text-cleaning pipeline built entirely from small, independently testable, pure functions, composed together with `pipe()`; this directly extends the NLP text-transformation examples from section 19.2 into a single, clearly named, reusable pipeline function.

22. Functional Pipelines

22.1 The Pipeline Concept

A pipeline is a sequence of transformation stages applied in order to data, where each stage's output becomes the next stage's input: Raw Data -> Clean -> Filter -> Transform -> Validate -> Aggregate -> Output. This section builds a complete, practical pipeline following exactly this shape, directly applying the composition tools from section 21 and the record-processing techniques from sections 17-19.

22.2 Defining Each Stage as a Pure Function

```
def clean(records):
    return [
        {**r, 'name': r['name'].strip(), 'amount': r['amount']}
        for r in records if r.get('name') is not None
    ]

def filter_valid(records):
    return [r for r in records if r['amount'] is not None and r['amount'] >= 0]

def transform(records):
    return [{**r, 'amount_with_tax': round(r['amount'] * 1.08, 2)} for r in records]

def validate(records):
    for r in records:
        if not r['name']:
            raise ValueError(f'Invalid record: {r}')
    return records

def aggregate(records):
    return {'count': len(records), 'total': sum(r['amount_with_tax'] for r in
records)}
```

Each stage takes the whole dataset and returns a new, transformed dataset (or, for aggregate, a final summary), following the immutable, non-mutating style established in section 13; every stage is independently readable, independently testable (section 26), and does exactly one clearly named job.

22.3 Composing the Full Pipeline

```
def pipe(*functions):
    from functools import reduce
    def piped(x):
        return reduce(lambda acc, f: f(acc), functions, x)
    return piped

run_pipeline = pipe(clean, filter_valid, transform, validate, aggregate)

raw_data = [
    {'name': ' Ada ', 'amount': 100},
    {'name': None, 'amount': 50},
```

```
{'name': 'Alan', 'amount': -10},  
{'name': 'Grace', 'amount': 200},  
]  
  
result = run_pipeline(raw_data)  
print(result)
```

Reusing `pipe()` from section 21.5, the entire Raw Data -> Clean -> Filter -> Transform -> Validate -> Aggregate -> Output flow becomes a single, declarative, top-to-bottom readable line: `run_pipeline = pipe(clean, filter_valid, transform, validate, aggregate)`. Every stage's order is explicit and easy to reorder, remove, or extend without touching the internal logic of any other stage.

22.4 Why This Makes Pipelines Modular and Testable

- Each stage can be unit tested completely in isolation, with a small, hand-crafted input and a known expected output, with no dependency on any other stage (fully demonstrated in section 26).
- Stages can be reordered, replaced, or removed by simply editing the `pipe(...)` call, without touching the internal implementation of any individual stage.
- A new stage (for example, a deduplication step) can be inserted anywhere in the sequence with a single line change.
- Because every stage is a pure function operating on, and returning, a complete dataset, the intermediate result after any stage can be inspected directly for debugging, simply by calling that one stage manually with the same input.

23. Lazy Evaluation

23.1 Lazy vs Eager Evaluation

Eager evaluation computes a result immediately, as soon as the expression producing it is encountered; a list comprehension is eager, it builds and stores the entire resulting list right away. Lazy evaluation defers computation until the result is actually needed, computing values one at a time, on demand, rather than all at once upfront. `map()`, `filter()`, and generator expressions are all lazy in Python, a property already demonstrated concretely in sections 5.11 and 15.5, and formalized completely here.

23.2 `map()` and `filter()` Revisited

```
numbers = [1, 2, 3, 4, 5]
lazy = map(lambda x: x ** 2, numbers)
eager = [x ** 2 for x in numbers]

print(type(lazy), type(eager))
```

Both ultimately can produce the same values, but lazy is a map object holding a not-yet-executed plan, while eager is already a fully realized, complete list sitting in memory; this distinction is the entire subject of this section.

23.3 Generator Expressions Revisited

```
large_range = range(10_000_000)
lazy_squares = (x ** 2 for x in large_range)
eager_squares = [x ** 2 for x in large_range]
```

Code Line	Explanation
<code>lazy_squares</code>	Creation is essentially instantaneous and uses a small, constant amount of memory, regardless of how large <code>large_range</code> is, since no squares have actually been computed or stored yet.
<code>eager_squares</code>	Creation immediately computes and stores all 10 million squared values in memory at once, which takes real, measurable time and a genuinely large amount of memory.

23.4 Iterators, Recapped Briefly

Both `map()/filter()` objects and generator expressions are examples of Python's iterator protocol: an object that produces its next value on demand via `__next__()`, and signals it is exhausted by raising `StopIteration` internally (handled automatically by a `for` loop or `list()/sum()/etc.`). This handbook assumes iterators and generators (via `yield` and generator functions) are covered in your fundamentals materials; this section focuses specifically on how laziness, a general concept, is expressed through them in a functional-programming context, not on iterator/generator mechanics themselves.

Distinguishing the Concepts: Lazy evaluation and iterators/generators are closely related but distinct ideas. Laziness is the general principle of deferring computation until needed. Iterators and generators are the specific Python language mechanisms that make lazy evaluation possible and practical. `map()`, `filter()`,

and generator expressions are all functional-programming-flavored applications built on top of the iterator protocol.

23.5 Memory Efficiency

```
import sys
numbers = range(1_000_000)

eager = [x for x in numbers]
lazy = (x for x in numbers)

print(sys.getsizeof(eager))
print(sys.getsizeof(lazy))
```

`sys.getsizeof(eager)` reports a size proportional to the number of elements stored (many megabytes for a million integers); `sys.getsizeof(lazy)` reports a small, essentially fixed size, since a generator only ever holds the state needed to produce its next value, not every value it will ever produce.

23.6 Streaming Data and Large Datasets

```
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

def parse_lines(lines):
    return (line.split(',') for line in lines)

def filter_valid(rows):
    return (row for row in rows if len(row) == 3)

pipeline = filter_valid(parse_lines(read_large_file('huge_data.csv')))
for row in pipeline:
    process(row)
```

Every stage of this pipeline, reading the file, splitting each line, filtering rows, is lazy; at no point does the entire file get loaded into memory at once, even though the pipeline conceptually applies three separate transformation steps. Only one line's worth of data is 'in flight' through the pipeline at any given moment, making this pattern capable of processing files far larger than available memory, a capability an eager, list-based approach fundamentally could not offer.

23.7 Why Lazy Evaluation Matters in Data Science and AI

- Datasets in real Data Science and AI work frequently exceed available memory; lazy pipelines process data in a memory-bounded, streaming fashion instead of requiring the whole dataset to fit at once.
- Lazy pipelines can begin producing usable output (and downstream processing can begin consuming it) before the entire input has even finished being read, reducing time-to-first-result for very large inputs.
- Infinite or continuously arriving data (a live sensor feed, a streaming API) can only be meaningfully processed lazily; an eager approach that tries to collect 'everything' first would simply never finish.

- Training and inference pipelines for large models very commonly use lazy, generator-based data loading specifically to avoid loading an entire, massive training set into memory before training can even begin.

24. Functional Programming with itertools

The itertools module is the standard library's toolkit of fast, memory-efficient, lazy iterator-building functions, a natural, deeply functional companion to `map()` and `filter()`. This section covers each major tool with a genuine practical use case, not just its signature.

24.1 chain()

```
from itertools import chain
batch_1 = [1, 2, 3]
batch_2 = [4, 5, 6]
combined = list(chain(batch_1, batch_2))
print(combined)
```

Lazily concatenates multiple iterables into one continuous stream, without first copying them into a single combined list; useful for processing several separate data sources (multiple files, multiple API pages) as if they were one unified sequence.

24.2 compress()

```
from itertools import compress
data = ['a', 'b', 'c', 'd']
selectors = [1, 0, 1, 0]
print(list(compress(data, selectors)))
```

Selects elements from data wherever the corresponding position in selectors is truthy; a practical use case is applying a previously computed boolean mask (from a validation step) to filter a parallel dataset.

24.3 count()

```
from itertools import count
for i in count(start=10, step=5):
    print(i)
    if i > 25:
        break
```

An infinite lazy counter, starting at start and increasing by step each time; useful for generating sequential IDs on demand, or as a building block combined with other itertools functions (like `takewhile`, section 24.7) that impose their own stopping condition.

24.4 cycle()

```
from itertools import cycle, islice
colors = cycle(['red', 'green', 'blue'])
first_seven = list(islice(colors, 7))
print(first_seven)
```

Infinitely repeats the elements of an iterable in a loop; practically useful for cyclically assigning a fixed set of labels, colors, or worker IDs across a longer sequence of items, one at a time. `islice()` (section 24.7) is required here to actually pull a finite number of values out, since `cycle()` alone never stops on its own.

24.5 repeat()

```
from itertools import repeat
print(list(repeat('x', 4)))
print(list(map(pow, range(5), repeat(2))))
```

Code Line	Explanation
<code>repeat('x', 4)</code>	Produces the same value a fixed number of times: ['x', 'x', 'x', 'x'].
<code>map(pow, range(5), repeat(2))</code>	A genuinely useful pattern: pairing <code>repeat(2)</code> with <code>map()</code> 's multiple-iterable behavior (section 5.4) supplies a constant second argument (here, exponent 2) to every call, computing each number in <code>range(5)</code> squared, without needing a lambda at all.

24.6 islice()

```
from itertools import islice
def read_large_file(path):
    with open(path) as f:
        for line in f:
            yield line
```

```
first_100_lines = list(islice(read_large_file('huge.log'), 100))
```

Slices a lazy iterator the way ordinary `[start:stop:step]` slicing works on a list, but without requiring the underlying iterator to support indexing or be fully materialized first; essential for peeking at just the first `n` items of a large or infinite lazy stream, directly extending the lazy-file-processing pattern from section 23.6.

24.7 takewhile() and dropwhile()

```
from itertools import takewhile, dropwhile
numbers = [2, 4, 6, 7, 8, 10]
print(list(takewhile(lambda x: x % 2 == 0, numbers)))
print(list(dropwhile(lambda x: x % 2 == 0, numbers)))
```

Code Line	Explanation
<code>takewhile(condition, iterable)</code>	Yields elements from the start of the iterable only as long as condition stays true, and stops permanently at the first element where it becomes false, even if later elements would have passed; here [2, 4, 6].
<code>dropwhile(condition, iterable)</code>	The inverse: skips elements from the start as long as condition is true, then yields every remaining element from that point onward, regardless of whether they individually satisfy the condition; here [7, 8, 10].

This is meaningfully different from `filter()`, which checks every single element independently; `takewhile/dropwhile` care specifically about a contiguous run from the beginning, useful for processing sorted or time-ordered data (for example, 'take readings while temperature stays below a threshold').

24.8 groupby()

```
from itertools import groupby
records = [
    {'dept': 'Eng', 'name': 'Ada'},
    {'dept': 'Eng', 'name': 'Grace'},
    {'dept': 'Sales', 'name': 'Alan'},
]

for dept, group in groupby(records, key=lambda r: r['dept']):
    print(dept, [r['name'] for r in group])
```

Code Line	Explanation
groupby(records, key=...)	Groups CONSECUTIVE elements that share the same key; this is a critical, frequently misunderstood detail.
The critical requirement	groupby() does NOT group elements from anywhere in the iterable the way a SQL GROUP BY or a pandas .groupby() does; it only groups adjacent runs. The input data must already be sorted by the grouping key beforehand, or elements with the same key that are not adjacent will incorrectly end up in separate groups.
Critical Requirement: Before using itertools.groupby(), sort the data by the same key first: records.sort(key=lambda r: r['dept']), or sorted_records = sorted(records, key=...). Forgetting this is one of the most common mistakes involving itertools, covered again explicitly in section 28.6.	

24.9 product(), permutations(), combinations()

```
from itertools import product, permutations, combinations

print(list(product([1, 2], ['a', 'b'])))
print(list(permutations([1, 2, 3], 2)))
print(list(combinations([1, 2, 3], 2)))
```

Code Line	Explanation
product([1, 2], ['a', 'b'])	The Cartesian product: every possible pair combining one element from each input, [(1,'a'), (1,'b'), (2,'a'), (2,'b')]; practically useful for generating every combination of hyperparameter values in a grid search, for example.
permutations([1, 2, 3], 2)	Every possible ORDERED arrangement of 2 elements chosen from the input, where order matters, so (1,2) and (2,1) are both included separately.
combinations([1, 2, 3], 2)	Every possible UNORDERED selection of 2 elements, where (1,2) and (2,1) are considered the same and only included once.

25. Functional Programming vs Traditional Loops

This section gives honest, side-by-side comparisons rather than declaring one style universally superior; the right choice depends on readability, the specific transformation, and team convention.

25.1 for Loop vs map()

```
numbers = [1, 2, 3, 4]

# Loop
squares = []
for n in numbers:
    squares.append(n ** 2)

# map()
squares = list(map(lambda n: n ** 2, numbers))

# Comprehension (often the most Pythonic middle ground)
squares = [n ** 2 for n in numbers]
```

Code Line	Explanation
Readability	The comprehension is generally considered the most directly readable of the three for this simple case; the loop is the most verbose but is immediately understandable to any Python programmer regardless of FP familiarity; map() sits in between, and is favored specifically when the transformation function is already named and reused.
Performance	Comprehensions and map() are both typically faster than a manual loop with repeated .append() calls, since the looping overhead is handled internally in optimized C code; the difference is rarely significant outside large-scale or performance-critical code.

25.2 for Loop vs filter()

```
# Loop
evens = []
for n in numbers:
    if n % 2 == 0:
        evens.append(n)

# filter()
evens = list(filter(lambda n: n % 2 == 0, numbers))

# Comprehension
evens = [n for n in numbers if n % 2 == 0]
```

The same overall tradeoffs apply as in section 25.1; the comprehension form is generally the most idiomatic default for a simple filter, while filter(named_function, ...) reads very clearly when the condition already has a meaningful, reusable name.

25.3 Loop vs reduce()

```
# Loop
total = 0
for n in numbers:
    total += n

# reduce()
from functools import reduce
total = reduce(lambda acc, n: acc + n, numbers, 0)

# The actually idiomatic choice for this specific case
total = sum(numbers)
```

This is the clearest case in this whole section where neither the loop nor `reduce()` is the best choice; `sum()` exists precisely for this purpose and should always be preferred for plain summation, directly reinforcing the guidance already given in section 7.10.

25.4 Mutable Accumulation vs Functional Transformation

```
# Mutable accumulation style
results = []
for record in records:
    if record['active']:
        results.append(transform(record))

# Functional transformation style
results = [transform(r) for r in records if r['active']]
```

Both produce an identical results list; the functional version avoids the mutable `results = []` / `.append()` bookkeeping entirely, expressing the same logic as a single, direct transformation of records into a new list. For a simple case like this, the functional/comprehension version is usually preferred; once the loop body needs several statements, side effects, or complex branching, an explicit loop often remains clearer.

25.5 A Balanced Conclusion

- For simple, single-expression transformations and filters, comprehensions are usually the most Pythonic and readable choice.
- For transformations already expressed as a named, reusable function, `map()`/`filter()` communicate intent directly and avoid re-deriving the logic inline.
- For genuine accumulation into something other than a simple `sum`/`min`/`max`, `reduce()` is appropriate, but should never replace Python's own specific aggregation built-ins.
- For multi-step logic, branching, or code with necessary side effects (logging, I/O), an explicit loop usually remains the clearest choice, and forcing it into a functional shape purely for style can actively hurt readability (expanded fully in sections 28 and 29).

Reinforcing This Handbook's Core Position: None of `map()`, `filter()`, `reduce()`, or comprehensions are unconditionally faster or better than a loop. Performance differences are usually small for realistic code sizes, and Pythonic style prioritizes whichever version most clearly communicates intent to the next reader, not adherence to a functional-programming rule for its own sake.

26. Functional Programming and Testing

26.1 Why Pure Functions Are Easier to Unit Test

A pure function's entire behavior is fully determined by its arguments, with no hidden dependencies to configure and no side effects to detect; a test for a pure function is simply a direct statement of 'given this input, expect this output', requiring no setup, no teardown, and no special test infrastructure to isolate it from the rest of the system.

26.2 A Practical Unit-Testing Example

```
def calculate_discount(price, percent):
    if not (0 <= percent <= 100):
        raise ValueError('percent must be between 0 and 100')
    return round(price * (1 - percent / 100), 2)

import unittest

class TestCalculateDiscount(unittest.TestCase):
    def test_no_discount(self):
        self.assertEqual(calculate_discount(100, 0), 100.0)

    def test_full_discount(self):
        self.assertEqual(calculate_discount(100, 100), 0.0)

    def test_partial_discount(self):
        self.assertEqual(calculate_discount(200, 25), 150.0)

    def test_invalid_percent_raises(self):
        with self.assertRaises(ValueError):
            calculate_discount(100, 150)
```

Code Line	Explanation
Why every test is trivial to write	Each test simply supplies fixed inputs and asserts the exact expected output; there is no mock object, no fixture, no database connection, and no cleanup step required anywhere, a direct, practical consequence of <code>calculate_discount()</code> being pure.
Compare to testing an impure function	A function reading a live database or the system clock would require mocking that dependency, setting up test data beforehand, and tearing it down afterward, all extra work and extra opportunities for a flawed test setup to produce a misleading result.

26.3 Easier Debugging

Directly connecting back to section 14.6: when a pure function misbehaves, the bug is guaranteed to be inside that function's own logic, since it has no hidden external state to blame. Reproducing the bug is as simple as calling the function again with the same input; there is no need to first recreate some particular external system state.

26.4 Reuse

A pure function's behavior does not depend on where or how it is called from; it can safely be reused across completely different parts of a codebase, or even copied into a different project entirely, with confidence that it will behave identically, since it carries no hidden coupling to its original calling context.

26.5 Reasoning About Code

Referential transparency (section 1.6) means a pure function call can be mentally substituted with its result when reading code, exactly like simplifying a mathematical expression; this significantly reduces the amount of surrounding context a reader needs to hold in their head to understand what a piece of code does, compared to an impure function whose result might depend on when, or how many times, or in what order it happens to be called.

26.6 Parallelization

Because a pure function's result never depends on anything beyond its own arguments, and it never modifies any shared state, many pure function calls can, in principle, be safely run in any order, or even simultaneously, without one call's execution interfering with another's result. This property is what makes pure functions a natural fit for certain kinds of parallel processing, a connection explored specifically, and with appropriate caution, in section 27.

27. Functional Programming and Concurrency

27.1 Why Statelessness Helps With Parallel Work

The core danger in running code concurrently (multiple pieces of work happening at overlapping times) is a race condition: two or more concurrent operations reading and writing the same shared, mutable state in an order that produces an incorrect or unpredictable result. A pure function, by definition, touches no shared mutable state at all (section 14), so calling it many times concurrently, each call operating on its own independent input, carries none of that risk; there is simply no shared state for the calls to race over.

27.2 A Practical multiprocessing Example

```
from multiprocessing import Pool

def process_record(record):
    return {**record, 'score': record['value'] * 1.1}

records = [{'value': v} for v in range(10)]

with Pool(processes=4) as pool:
    results = pool.map(process_record, records)

print(results)
```

Code Line	Explanation
Why process_record() is safe to parallelize	It is pure: given one record, it always returns the same new, independent result, with no reference to or modification of any state shared with other calls; running it on 10 different records across 4 separate processes at once produces the exact same results as running it 10 times sequentially, just faster.
pool.map()	Directly mirrors the built-in map()'s signature and behavior (section 5), but distributes the work across multiple separate processes rather than a single one; this parallel structure is only safe and correct here specifically because process_record() is pure.

27.3 Batch and Parallel Processing

```
from multiprocessing import Pool

def process_batch(batch):
    return [process_record(r) for r in batch]

def batch(iterable, size):
    items = list(iterable)
    return [items[i:i + size] for i in range(0, len(items), size)]

batches = batch(records, size=3)
with Pool(processes=4) as pool:
    batch_results = pool.map(process_batch, batches)
```

Combining the batching function from section 18.5 with multiprocessing.Pool applies the pure per-record transformation across several worker processes at once, a realistic pattern for parallelizing a large data-processing or feature-engineering job (section 18.1) across available CPU cores.

27.4 Important Honest Caveats

- Functional programming does NOT automatically make Python code parallel; map(), filter(), and comprehensions all still run sequentially, on a single thread, by default. Genuine parallelism requires explicitly using a tool like multiprocessing, concurrent.futures, or an async framework.
- Python's Global Interpreter Lock (GIL) means ordinary threading does not achieve true CPU-bound parallelism for pure Python code; multiprocessing (separate processes, as shown above) is typically required for CPU-bound work to actually run in parallel, at the cost of process-creation overhead and the requirement that data passed between processes be serializable (picklable).
- Purity makes parallelization SAFER and easier to reason about correctly; it does not, by itself, make code faster or automatically parallel. The developer must still explicitly choose and correctly apply a concurrency tool.
- I/O-bound work (waiting on network requests or file operations) benefits from a different set of tools entirely (asyncio, threading), where purity matters less than correctly managing the waiting itself; this handbook's focus remains on the data-transformation, CPU-bound use case where purity's safety benefit is most directly relevant.

Accuracy Note: This section deliberately avoids the common but inaccurate claim that 'functional programming makes Python parallel'. The honest, precise claim is narrower and still valuable: pure, stateless functions are SAFE to parallelize without introducing race conditions, which removes a major source of risk and complexity when you do explicitly choose to parallelize your code with a tool like multiprocessing.

28. Common Mistakes

28.1 Overusing Lambda

```
# Overused: a multi-purpose, unclear inline lambda
process = lambda x: x.strip().lower().replace(' ', '_') if isinstance(x, str) else
str(x)

# Better: a named function with a clear purpose
def normalize_key(x):
    if isinstance(x, str):
        return x.strip().lower().replace(' ', '_')
    return str(x)
```

As first cautioned in section 4.15, once a lambda's logic grows beyond a single, obviously simple expression, it stops improving readability and starts hurting it; the named `def` version is both easier to read and, unlike the lambda, can be given a proper docstring and a meaningful, searchable name.

28.2 Overusing `reduce()`

```
# Overused: reduce() for something a built-in already does
from functools import reduce
total = reduce(lambda acc, x: acc + x, numbers, 0)
maximum = reduce(lambda acc, x: x if x > acc else acc, numbers)

# Better
total = sum(numbers)
maximum = max(numbers)
```

Directly reinforcing sections 7.10 and 25.3: `reduce()` should be reserved for accumulation logic with no direct built-in equivalent; using it for a plain sum or max only adds unnecessary indirection for a reader to untangle.

28.3 Writing Unreadable Chains

```
# Unreadable: everything crammed into one line
result = list(map(lambda x: x['total'], filter(lambda x: x['active'], map(lambda x:
{**x, 'total': x['price'] * x['qty']}, records))))

# Better: broken into named, sequential steps
with_totals = [{**r, 'total': r['price'] * r['qty']} for r in records]
active_only = [r for r in with_totals if r['active']]
totals = [r['total'] for r in active_only]
```

The first version requires reading from the inside out, tracking three nested lambdas and their arguments simultaneously, to understand what happens first. The second version reads top to bottom in the actual order of execution, with each intermediate result given a clear name; this is very often a worthwhile tradeoff even though it is more lines of code, directly connecting to the comprehension-complexity guidance in section 15.10.

28.4 Excessive Function Nesting

```
# Excessive: composition nested many levels deep, hard to trace
```

```
result = f(g(h(i(j(k(x))))))
```

Beyond two or three levels, manually nested function calls become genuinely difficult to read and debug, since there is no way to inspect an intermediate value without restructuring the code; prefer the `pipe()/compose()` utilities from section 21, or simply intermediate named variables, once nesting grows beyond a couple of levels.

28.5 Hidden Side Effects

```
processed_count = 0

def process(item):
    global processed_count
    processed_count += 1
    return item.upper()

results = list(map(process, items))
```

`process()` looks like an innocent, pure transformation used with `map()`, but it secretly mutates a global counter as a side effect; anyone reading `results = list(map(process, items))` alone would have no reason to suspect `processed_count` is also being changed. This directly violates the purity principles from section 14 and makes the code's true behavior invisible from its call site.

28.6 Mutating Shared State

```
# Dangerous: the lambda mutates a list shared across calls
shared_log = []
log_and_double = lambda x: (shared_log.append(x), x * 2)[1]

results = list(map(log_and_double, [1, 2, 3]))
```

Packing a side effect (appending to `shared_log`) inside a lambda used with `map()`, via the `(side_effect(), actual_value)[1]` trick, is a clear anti-pattern: it hides a mutation inside what should be a pure transformation, makes the code fragile and confusing, and should always be replaced with an explicit loop if logging or side effects are genuinely needed.

28.7 Confusing Lazy Objects With Lists

```
squares = map(lambda x: x ** 2, [1, 2, 3])
print(len(squares))
```

Code Line	Explanation
The error	<code>TypeError: object of type 'map' has no len()</code>
Why	A map object (like a filter object or a generator) is a lazy iterator, not a list; it has no defined length until it is actually consumed, and it does not support indexing, <code>len()</code> , or being iterated over more than once. This is a frequent beginner trap when first learning <code>map()/filter()</code> after being accustomed to lists (section 5.2 and section 23 cover this laziness formally).
The fix	Wrap it in <code>list()</code> first if a genuine list (with a length, repeatable iteration, and indexing) is actually needed: <code>squares_list = list(map(...))</code> .

28.8 Incorrect Closure Behavior (The Classic Late-Binding Trap)

```
functions = []
for i in range(3):
    functions.append(lambda: i)

print([f() for f in functions])
```

Code Line	Explanation
Expected (by many beginners)	[0, 1, 2], one function per loop iteration's value of i.
Actual output	[2, 2, 2]. Every lambda captures the SAME variable i, not its value at the time the lambda was created; by the time any of the lambdas are actually called, the loop has finished, and i holds its final value, 2, for all three.
The fix	Force the current value to be captured immediately by giving it a default argument: <code>functions.append(lambda i=i: i)</code> ; default argument values ARE evaluated immediately at function-definition time, capturing the current value of i at that specific point in the loop, unlike the closure's captured variable itself, which is only looked up when the function is actually called.

Classic Interview Topic: This late-binding closure trap is one of the most well-known and frequently asked-about gotchas in all of Python, and it is directly related to the closure mechanics explained in section 12: a closure remembers the VARIABLE, not a frozen snapshot of its value at creation time.

28.9 Poor Use of partial()

```
from functools import partial

# Poor: fixing an argument that actually varies on every call, defeating the purpose
get_user = partial(fetch_from_api, endpoint='users', user_id=42)
get_user(user_id=99) # confusingly still works via override, but the original
intent is unclear
```

`partial()` is meant to fix arguments that genuinely stay constant across many calls (like `base_url` or `api_key` in section 11.5); fixing an argument that actually needs to vary per call, and then relying on it being overridden later, defeats the purpose and creates confusing, misleading code about what is actually 'fixed'.

28.10 Sacrificing Readability for 'Functional Purity'

```
# Overly 'clever': avoiding a simple loop purely to feel functional
from functools import reduce
result = reduce(
    lambda acc, x: acc + [x * 2] if x % 2 == 0 else acc,
    numbers,
    []
)

# Clearer, equally 'functional' in spirit, far more readable
result = [x * 2 for x in numbers if x % 2 == 0]
```

This is the single most important mistake in this entire section, and the thesis this whole handbook keeps returning to: functional programming is a set of tools for writing clearer, more predictable code, not a set of rules to follow for their own sake. Choosing `reduce()` over a comprehension specifically to avoid looking 'imperative' produces objectively worse code here; Pythonic readability should always outrank dogmatic adherence to any single paradigm, a position formalized as this handbook's central professional guidance in section 29.

29. Professional Best Practices

This section consolidates a decision framework for choosing among the tools covered throughout this handbook, prioritizing Pythonic readability and real-world clarity over any rule that functional style is inherently preferable.

29.1 When to Use `def`

- The logic requires more than one expression, statements, loops, or exception handling.
- The function is reused in more than one place, or benefits from a clear, searchable name and a docstring.
- The function is public API, part of a module's or class's documented interface.

29.2 When to Use `lambda`

- A genuinely trivial, single-expression transformation used exactly once, inline, as an argument to `map()`, `filter()`, `sorted(key=)`, `min()`, or `max()`.
- Never assign a `lambda` to a variable purely to name it; use `def` instead (section 4.15).

29.3 When to Use Comprehensions

- The default, generally preferred choice for straightforward transform-and/or-filter operations that fit comfortably and clearly on one or two lines.
- Switch to an explicit loop once a comprehension would require nested conditionals, multiple for clauses beyond a simple flatten, or logic that is hard to read at a glance (section 15.10).

29.4 When to Use `map()` and `filter()`

- When the transformation or filtering function already exists as a named, reusable function, avoiding re-deriving its logic inline as a comprehension expression.
- Be mindful that both return lazy objects (section 28.7); wrap in `list()` explicitly when a concrete, reusable, indexable collection is actually needed.

29.5 When to Use `reduce()`

- Only when the accumulation genuinely produces something a specific built-in (`sum`, `min`, `max`, `any`, `all`) does not already provide directly (section 7.11).
- Prefer a named accumulator function over a dense inline `lambda` once the reduction logic is non-trivial.

29.6 When to Use Generators

- Whenever the full result set does not need to exist in memory all at once, especially for large or streaming data (section 23.6).
- When a computation might be short-circuited early (for example, finding the first matching item), avoiding unnecessary work on the remaining elements.

29.7 When to Use Higher-Order Functions

- When behavior itself genuinely needs to be configurable or swappable at runtime (a strategy pattern, a pluggable validation rule, a pipeline stage).
- When building reusable infrastructure, like decorators, composition utilities, or pipeline runners, that is meant to work with many different underlying functions.

29.8 When to Use `partial()`

- When several arguments to a function are genuinely fixed and constant across many calls in a given context (section 11), producing a clearer, more specialized callable than repeatedly passing the same arguments manually.
- Avoid it when the 'fixed' argument actually needs to vary per call (section 28.9).

29.9 When to Use Function Composition

- When a sequence of independent, single-purpose transformation steps needs to be expressed and reused as one clearly named unit (sections 21-22).
- Prefer a small number of well-named stages over composing many tiny, cryptically named functions, which can become as hard to trace as deeply nested calls (section 28.4).

29.10 The Overarching Principle

Every guideline in this section serves one goal: code that a competent Python developer, including your own future self, can read, understand, and safely modify with minimal friction. Functional programming tools in Python exist to serve that goal, not to be applied as an ideology. When a functional technique genuinely makes code clearer, more predictable, or more testable, as it very often does for data transformation logic, use it confidently. When it does not, an ordinary loop, an if/else block, or a straightforward class is not a failure to 'be functional'; it is simply the right, Pythonic tool for that particular job.

30. Real-World Projects

Eight complete projects, increasing in difficulty, each applying the functional programming techniques from this handbook to a realistic scenario. Every project follows the same structure: problem statement, requirements, concepts tested, a step-by-step solution, complete code, a code explanation, expected output, and possible improvements.

30.1 Project 1: Student Score Transformation System

Problem Statement

Given a list of student score records, compute a letter grade, a pass/fail status, and a class ranking for each student, using only functional-style transformations.

Requirements

- Input: a list of dicts with 'name' and 'score' (0-100).
- Output: each record enriched with 'grade', 'passed', and 'rank' (1 = highest score).

Concepts Tested

map(), lambda, sorted() with key=, operator.itemgetter, pure functions, immutable transformation.

Step-by-Step Solution

1. Write a pure grade(score) function.
2. Use map() to attach grade and passed to every record.
3. Sort a copy of the enriched records by score descending to determine rank.
4. Merge the computed rank back into each original record without mutating anything in place.

```
from operator import itemgetter

def grade(score):
    if score >= 90: return 'A'
    if score >= 80: return 'B'
    if score >= 70: return 'C'
    if score >= 60: return 'D'
    return 'F'

def enrich(record):
    return {**record, 'grade': grade(record['score']), 'passed': record['score'] >= 60}

def add_ranks(records):
    ranked_order = sorted(records, key=itemgetter('score'), reverse=True)
    rank_by_name = {r['name']: i + 1 for i, r in enumerate(ranked_order)}
    return [{**r, 'rank': rank_by_name[r['name']]} for r in records]

students = [
    {'name': 'Ada', 'score': 92},
```

```

    {'name': 'Alan', 'score': 58},
    {'name': 'Grace', 'score': 78},
]

result = add_ranks(list(map(enrich, students)))
for r in result:
    print(r)

```

Code Explanation

`grade()` is a small pure function tested trivially in isolation. `enrich()` uses `map()`-friendly dict-merging (the `**r, ...` pattern from section 5.8) to add two derived fields without mutating the input. `add_ranks()` computes rank separately, via a sorted copy and a name-to-rank lookup dictionary, then merges that rank back in, keeping every step a pure, traceable transformation of the previous step's output.

Expected Output

```

{'name': 'Ada', 'score': 92, 'grade': 'A', 'passed': True, 'rank': 1}
{'name': 'Alan', 'score': 58, 'grade': 'F', 'passed': False, 'rank': 3}
{'name': 'Grace', 'score': 78, 'grade': 'C', 'passed': True, 'rank': 2}

```

Possible Improvements

- Handle tied scores with proper tie-aware ranking (e.g., dense ranking) instead of a strict sequential rank.
- Compose `enrich` and `add_ranks` with the `pipe()` utility from section 21.5.

30.2 Project 2: Sales Data Processing Pipeline

Problem Statement

Build a functional pipeline that cleans raw sales records, computes line totals, and aggregates total revenue per product.

Requirements

- Reject records with a missing or non-positive quantity or price.
- Compute a 'total' field per record.
- Produce a dictionary of total revenue per product.

Concepts Tested

`filter()`, `map()`, `reduce()`, `pipe()`-style composition, pure functions.

Step-by-Step Solution

1. Define a validity filter.
2. Map a total-computing transform over valid records.
3. Reduce the transformed records into a per-product revenue dictionary.

```

from functools import reduce

```

```
def is_valid(r):
    return r.get('price', 0) > 0 and r.get('qty', 0) > 0

def with_total(r):
    return {**r, 'total': round(r['price'] * r['qty'], 2)}

def aggregate_by_product(records):
    return reduce(
        lambda acc, r: {**acc, r['product']: acc.get(r['product'], 0) + r['total']},
        records, {}
    )

raw_sales = [
    {'product': 'Widget', 'price': 9.99, 'qty': 3},
    {'product': 'Widget', 'price': 9.99, 'qty': 0},
    {'product': 'Gadget', 'price': 19.99, 'qty': 2},
]

valid = list(filter(is_valid, raw_sales))
totalled = list(map(with_total, valid))
revenue = aggregate_by_product(totalled)
print(revenue)
```

Code Explanation

Each stage (filter -> map -> reduce) mirrors the pipeline shape from section 22: filtering removes invalid rows first, mapping derives the per-row total, and reduce() folds the results into a single aggregated dictionary, directly reusing the record-combining reduce() pattern from section 7.8.

Expected Output

```
{'Widget': 29.97, 'Gadget': 39.98}
```

Possible Improvements

- Return which records were rejected and why, for a data-quality report.
- Swap the pure-Python aggregation for pandas groupby once volume grows large (per section 17.8's honest guidance).

30.3 Project 3: Customer Data Cleaning System

Problem Statement

Clean a raw customer dataset containing inconsistent casing, whitespace, and missing/invalid email addresses.

Requirements

- Trim and title-case names; lowercase emails.
- Discard records with an invalid email (no '@').
- Deduplicate by email, keeping the first occurrence.

Concepts Tested

map(), filter(), closures (for a stateful-but-encapsulated dedup helper), pure vs impure design.

Step-by-Step Solution

- 1. Normalize every record with map().
- 2. Filter out records with an invalid email.
- 3. Deduplicate using a closure-based 'seen' tracker, deliberately isolating the one necessary piece of mutable state.

```
def normalize(r):
    return {
        'name': r.get('name', '').strip().title(),
        'email': (r.get('email') or '').strip().lower(),
    }

def is_valid_email(r):
    return '@' in r['email']

def make_deduplicator():
    seen = set()
    def is_new(r):
        if r['email'] in seen:
            return False
        seen.add(r['email'])
        return True
    return is_new

raw_customers = [
    {'name': ' ada ', 'email': 'ADA@EXAMPLE.COM'},
    {'name': 'Alan', 'email': 'not-an-email'},
    {'name': 'ada duplicate', 'email': 'ada@example.com'},
]

cleaned = list(filter(is_valid_email, map(normalize, raw_customers)))
dedup = make_deduplicator()
unique = list(filter(dedup, cleaned))
print(unique)
```

Code Explanation

normalize() and is_valid_email() are pure. Deduplication genuinely requires memory of what has been seen before, an inherently stateful operation, so rather than reaching for a global variable (section 13.3's warning), that single piece of necessary state is deliberately isolated inside a closure (section 12), keeping its scope minimal and explicit.

Expected Output

```
[{'name': 'Ada', 'email': 'ada@example.com'}]
```

Possible Improvements

- Report which records were rejected as duplicates or invalid, rather than silently dropping them.
- Validate email format more rigorously with a regular expression.

30.4 Project 4: Financial Transaction Analyzer

Problem Statement

Analyze a list of bank transactions to compute total credits, total debits, net balance change, and flag any suspicious large transactions.

Requirements

- A transaction is 'suspicious' if its absolute amount exceeds a configurable threshold.
- Report totals and a list of suspicious transactions.

Concepts Tested

filter(), reduce(), partial(), function composition.

Step-by-Step Solution

- 1. Split transactions into credits and debits with filter().
- 2. Sum each group.
- 3. Use partial() to create a configurable 'is_suspicious' filter bound to a specific threshold.

```
from functools import partial

def is_suspicious(threshold, txn):
    return abs(txn['amount']) > threshold

def analyze(transactions, threshold=1000):
    credits = list(filter(lambda t: t['amount'] > 0, transactions))
    debits = list(filter(lambda t: t['amount'] < 0, transactions))
    flagged = list(filter(partial(is_suspicious, threshold), transactions))
    return {
        'total_credits': sum(t['amount'] for t in credits),
        'total_debits': sum(t['amount'] for t in debits),
        'net_change': sum(t['amount'] for t in transactions),
        'suspicious': flagged,
    }

transactions = [
    {'id': 1, 'amount': 500},
    {'id': 2, 'amount': -1200},
    {'id': 3, 'amount': -75},
]

print(analyze(transactions, threshold=1000))
```

Code Explanation

`partial(is_suspicious, threshold)` fixes the `threshold` argument, producing a single-argument predicate compatible with `filter()`, directly applying the partial-application technique from section 11 to build a configurable filter condition on demand.

Expected Output

```
{'total_credits': 500, 'total_debits': -1275, 'net_change': -775,
 'suspicious': [{'id': 2, 'amount': -1200}]}
```

Possible Improvements

- Support per-account thresholds instead of one global threshold.
- Add a running-balance calculation using a scan-like `reduce()` that keeps every intermediate total, not just the final one.

30.5 Project 5: Log-Processing Pipeline

Problem Statement

Process a large, potentially huge log file lazily, extracting and counting ERROR-level entries by their error code, without loading the entire file into memory.

Requirements

- Stream the file line by line.
- Parse and filter for ERROR lines only.
- Count occurrences per error code.

Concepts Tested

Lazy evaluation, generators, `itertools`, `reduce()`.

Step-by-Step Solution

1. A generator yields raw lines lazily from the file.
2. A generator expression filters for ERROR lines and parses out the error code.
3. `reduce()` (or `Counter`, noted as an improvement) tallies counts per code.

```
import re
from functools import reduce

def read_lines(path):
    with open(path) as f:
        for line in f:
            yield line.strip()

def extract_error_codes(lines):
    pattern = re.compile(r'ERROR \[(\w+)\]')
    for line in lines:
```

```

        match = pattern.search(line)
        if match:
            yield match.group(1)

def count_codes(codes):
    return reduce(lambda acc, code: {**acc, code: acc.get(code, 0) + 1}, codes, {})

# error_counts = count_codes(extract_error_codes(read_lines('server.log')))
```

Code Explanation

Every stage is lazy: `read_lines()` yields one line at a time, `extract_error_codes()` processes and yields only matching codes one at a time, and only `count_codes()` (via `reduce()`) actually forces full consumption, at which point the whole file has been streamed through in constant memory, directly applying the streaming pattern from section 23.6.

Expected Output (illustrative, given a matching log file)

```
{'DB001': 12, 'AUTH03': 4, 'NET500': 1}
```

Possible Improvements

- Use `collections.Counter` instead of a manual `reduce()` for a more idiomatic and slightly faster tally.
- Add `itertools.islice()` to process only the most recent N lines of a huge log for a quick sample.

30.6 Project 6: Data Preprocessing Pipeline

Problem Statement

Build a general-purpose, reusable preprocessing pipeline utility that can clean, impute missing values, and scale a numeric dataset, with each step swappable.

Requirements

- Support an ordered list of pure transformation stages.
- Provide default clean/impute/scale stages, but allow custom ones.

Concepts Tested

Function composition, higher-order functions, `pipe()`.

Step-by-Step Solution

```

from functools import reduce

def pipe(*functions):
    def piped(x):
        return reduce(lambda acc, f: f(acc), functions, x)
    return piped

def remove_none(values):
    return [v for v in values if v is not None]
```

```
def impute_mean(values):
    mean = sum(v for v in values if v is not None) / len(remove_none(values))
    return [v if v is not None else mean for v in values]

def min_max_scale(values):
    lo, hi = min(values), max(values)
    return [(v - lo) / (hi - lo) for v in values]

preprocess = pipe(impute_mean, min_max_scale)
raw = [10, None, 30, 50, None]
print(preprocess(raw))
```

Code Explanation

Each stage is a pure function operating on the whole list and returning a new list, so `pipe()` (section 21.5) can chain any subset or reordering of them with zero changes to the stages themselves; swapping `min_max_scale` for a z-score standardization function, for example, requires editing only the `pipe(...)` call.

Expected Output

```
[0.0, 0.25, 0.5, 1.0, 0.25]
```

Possible Improvements

- Make each stage log its before/after row count or summary statistics, as in the Data Cleaning Pipeline project from the companion Exception Handling handbook.
- Validate stage compatibility (e.g., scaling requires no remaining None values) before running the pipeline.

30.7 Project 7: Machine-Learning Feature Transformation Pipeline

Problem Statement

Transform a raw dataset of user records into a numeric feature matrix suitable for a model, including one-hot encoding a categorical column and deriving a new feature.

Requirements

- One-hot encode a 'plan' categorical field.
- Derive 'tenure_years' from a 'signup_date'.
- Output a list of flat numeric feature dicts.

Concepts Tested

`map()`, function composition, closures (for a fitted encoder), pure functions.

```
from datetime import date

def make_one_hot_encoder(categories):
    def encode(value):
        return {f'plan_{c}': int(value == c) for c in categories}
```

```

    return encode

def add_tenure(record, today=None):
    today = today or date.today()
    signup = date.fromisoformat(record['signup_date'])
    years = round((today - signup).days / 365, 2)
    return {**record, 'tenure_years': years}

def build_features(records, encoder):
    def to_features(r):
        r = add_tenure(r)
        return {'tenure_years': r['tenure_years'], **encoder(r['plan'])}
    return list(map(to_features, records))

users = [
    {'plan': 'pro', 'signup_date': '2023-01-15'},
    {'plan': 'free', 'signup_date': '2024-06-01'},
]

encoder = make_one_hot_encoder(['free', 'pro', 'enterprise'])
features = build_features(users, encoder)
for f in features:
    print(f)

```

Code Explanation

`make_one_hot_encoder()` is a closure factory (section 12.3) that 'fits' the encoder to a fixed set of known categories once, returning a reusable, consistent encoding function, exactly the kind of pattern needed so training and later inference-time encoding stay consistent. `build_features()` composes the tenure derivation and encoding into one clean, per-record transformation applied with `map()`.

Expected Output

```

{'tenure_years': ..., 'plan_free': 0, 'plan_pro': 1, 'plan_enterprise': 0}
{'tenure_years': ..., 'plan_free': 1, 'plan_pro': 0, 'plan_enterprise': 0}

```

Possible Improvements

- Persist the fitted encoder's category list so training-time and inference-time encoding always stay identical (directly connecting to the 'unseen category' problem discussed in your Exception Handling notes' ML section).
- Convert the final list of dicts into a numpy array or pandas DataFrame for direct model consumption.

30.8 Project 8: AI/NLP Text-Processing Pipeline

Problem Statement

Build a reusable text-preprocessing pipeline for an NLP task: normalize, tokenize, remove stopwords, and compute simple word-frequency features, entirely through composed pure functions.

Requirements

- Lowercase, strip punctuation, tokenize, remove stopwords.
- Compute top-N most frequent remaining tokens per document.

Concepts Tested

Composition/pipe(), map(), filter(), reduce(), sorted() with key=, closures.

```
import re
from functools import reduce

STOPWORDS = {'the', 'a', 'an', 'is', 'in', 'of', 'to', 'and'}

def normalize(text):
    return re.sub(r'[^a-z0-9\s]', '', text.lower())

def tokenize(text):
    return text.split()

def remove_stopwords(tokens):
    return [t for t in tokens if t not in STOPWORDS]

def pipe(*functions):
    def piped(x):
        return reduce(lambda acc, f: f(acc), functions, x)
    return piped

preprocess = pipe(normalize, tokenize, remove_stopwords)

def top_n_frequencies(tokens, n=3):
    counts = reduce(lambda acc, t: {**acc, t: acc.get(t, 0) + 1}, tokens, {})
    return sorted(counts.items(), key=lambda item: item[1], reverse=True)[:n]

document = 'The quick brown fox jumps over the lazy dog. The dog barks.'
tokens = preprocess(document)
print(tokens)
print(top_n_frequencies(tokens))
```

Code Explanation

preprocess is a single composed function built from three small, independently testable pure stages, directly reusing the pipe() utility from section 21.5. top_n_frequencies() reuses the reduce()-based tallying pattern from Project 5 and section 7.8, then applies sorted() with key=itemgetter-equivalent lambda logic (section 8) to rank and slice the top results.

Expected Output

```
['quick', 'brown', 'fox', 'jumps', 'over', 'lazy', 'dog', 'dog', 'barks']
[('dog', 2), ('quick', 1), ('brown', 1)]
```

Possible Improvements

- Replace the manual `reduce()`-based counter with `collections.Counter` for both clarity and performance.
- Extend the pipeline with stemming/lemmatization as an additional composable stage, without altering any existing stage.

31. Interview Preparation

50 questions spanning conceptual understanding, code-output prediction, debugging, comparative 'difference between' questions, scenarios, and Data Science/AI-specific applications. Every question includes a complete answer.

31.1 Conceptual Questions (1-10)

Q1. What does it mean for functions to be 'first-class' in Python?

Answer: It means functions are treated as ordinary values: they can be assigned to variables, passed as arguments, returned from other functions, and stored in data structures, exactly like an int or a list. See section 2.

Q2. What is a pure function? Give an example and a counter-example.

Answer: A pure function always returns the same output for the same input and has no observable side effects. Example: `def add(a, b): return a + b`. Counter-example: a function that appends to a global list or reads the current time. See section 14.

Q3. What is referential transparency?

Answer: An expression is referentially transparent if it can be replaced by its computed value anywhere it appears without changing the program's behavior; this holds for pure function calls but not for calls with side effects. See section 1.6.

Q4. Why is `reduce()` not a built-in function in Python 3?

Answer: It was deliberately moved to `functools` because it is often harder to read than an explicit loop or a more specific built-in (`sum`, `min`, `max`), and Python's design philosophy nudges developers toward considering clearer alternatives first. See section 7.1.

Q5. What is a closure?

Answer: A function object that remembers and can access variables from its enclosing lexical scope, even after that enclosing function has finished executing. See section 12.2.

Q6. Why does functional programming favor immutability?

Answer: Because mutable shared state is a major source of bugs (unexpected changes made elsewhere in a program) and makes code harder to reason about; immutability guarantees a value cannot change out from under code that assumes it is stable. See section 1.8 and 13.2.

Q7. What is lazy evaluation, and which Python constructs exhibit it?

Answer: Lazy evaluation defers computing a value until it is actually needed, rather than computing it immediately. `map()`, `filter()`, and generator expressions are all lazy in Python. See section 23.

Q8. What is function composition?

Answer: Combining two or more functions so the output of one becomes the input of the next, producing a single new function representing the whole chain: $(f \circ g)(x) = f(g(x))$. See section 21.1.

Q9. Does functional programming automatically make Python code run in parallel?

Answer: No. Purity makes code SAFE to parallelize (no shared mutable state to race over), but actual parallel execution still requires explicitly using a tool like multiprocessing or concurrent.futures. See section 27.4.

Q10. What is the difference between a higher-order function and an ordinary function?

Answer: A higher-order function accepts a function as an argument, returns a function, or both. An ordinary function only operates on plain data. See section 3.5.

31.2 Code-Output Questions (11-20)**Q11. What does this print?**

```
print(list(map(lambda x: x * 2, [1, 2, 3])))
```

Answer: [2, 4, 6]

Q12. What does this print?

```
nums = map(lambda x: x + 1, [1, 2, 3])
print(list(nums))
print(list(nums))
```

Answer: [2, 3, 4] then []. A map object is a one-time-use iterator; once fully consumed by the first list(), nothing remains for the second call. See section 5.10.

Q13. What does this print?

```
from functools import reduce
print(reduce(lambda acc, x: acc + x, [], 10))
```

Answer: 10. With an empty iterable and an explicit initializer, reduce() simply returns the initializer unchanged, since there is nothing to fold into it. See section 7.2.

Q14. What does this print?

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)
print([f() for f in funcs])
```

Answer: [2, 2, 2]. The classic late-binding closure trap: every lambda captures the same variable i, whose final value after the loop is 2. See section 28.8.

Q15. What does this print?

```
print(list(filter(None, [0, 1, '', 'a', [], [1], False, True])))
```

Answer: [1, 'a', [1], True]. filter(None, ...) keeps only truthy elements. See section 6.8.

Q16. What does this print?

```
square = lambda x: x ** 2
print(square)
```

Answer: Something like <function <lambda> at 0x...>. A lambda not called simply prints its function object representation, not a computed value. See section 4.16.

Q17. What does this print?

```
from functools import partial
def power(base, exp): return base ** exp
square = partial(power, exp=2)
print(square(5))
```

Answer: 25. exp is fixed to 2 by partial(); square(5) calls power(5, exp=2). See section 11.2.

Q18. What does this print?

```
print(sorted([('b', 2), ('a', 3), ('c', 1)], key=lambda t: t[1]))
```

Answer: [('c', 1), ('b', 2), ('a', 3)]. Sorted by each tuple's second element ascending. See section 8.5.

Q19. What does this print?

```
def outer():
    x = 10
    def inner():
        x = x + 1
        return x
    return inner
outer()
```

Answer: Raises UnboundLocalError. Assigning to x inside inner() (x = x + 1) makes x local to inner(), so the read on the right-hand side happens before any local x has been assigned; nonlocal x is required for this to work, per section 12.4.

Q20. What does this print?

```
from itertools import islice, count
print(list(islice(count(1), 5)))
```

Answer: [1, 2, 3, 4, 5]. count(1) lazily produces 1, 2, 3, ... forever; islice takes just the first 5. See sections 24.3 and 24.6.

31.3 Debugging Questions (21-28)

Q21. This is supposed to double every number, but raises a TypeError. Why?

```
numbers = [1, 2, 3]
doubled = map(lambda x: x * 2, numbers)
print(doubled[0])
```

Answer: A map object does not support indexing. Fix: wrap it in list() first, doubled = list(map(...)), then doubled[0] works. See section 28.7.

Q22. Why does this always print the same value for every key?

```
handlers = {}
for name in ['a', 'b', 'c']:
    handlers[name] = lambda: name
print([h() for h in handlers.values()])
```

Answer: ['c', 'c', 'c'] -- the same late-binding closure trap as Q14. Fix with a default argument: lambda name=name: name. See section 28.8.

Q23. Why does this reduce() call raise a TypeError?

```
from functools import reduce
reduce(lambda acc, x: acc + x, [])
```

Answer: With no initializer and an empty iterable, reduce() has nothing to use as a starting accumulator and nothing to combine, so it raises TypeError: reduce() of empty iterable with no initial value. Fix: supply an initializer, e.g. reduce(..., [], 0). See section 7.5.

Q24. This filter is supposed to remove empty strings but keeps them. Find the bug.

```
words = ['', 'hello', '', 'world']
result = list(filter(lambda w: w == '', words))
```

Answer: The condition is backward: w == "" KEEPS empty strings, since filter() keeps elements where the function is truthy. Fix: lambda w: w != "". See section 6.1.

Q25. Why does groupby() here produce three separate 'Eng' groups instead of one?

```
from itertools import groupby
records = [{'dept': 'Eng'}, {'dept': 'Sales'}, {'dept': 'Eng'}]
for k, g in groupby(records, key=lambda r: r['dept']):
    print(k, list(g))
```

Answer: itertools.groupby() only groups CONSECUTIVE matching elements; the data must be sorted by the key first. Fix: sort with records.sort(key=lambda r: r['dept']) before calling groupby(). See section 24.8.

Q26. This partial() call does not behave as expected. Why?

```
from functools import partial
def greet(greeting, name): return f'{greeting}, {name}'
hello = partial(greet, name='Ada')
```

```
print(hello('Hi'))
```

Answer: Raises `TypeError`: multiple values for argument 'name'. name was fixed by keyword, but 'Hi' is then supplied positionally, filling the first positional slot (greeting) correctly, but Python still tries to also bind it as name is already fixed -- actually the real issue is 'Hi' fills greeting fine; the error instead comes from calling `hello()` with no override needed. The fix depends on intent: call `hello(greeting='Hi')` explicitly, or fix greeting instead of name if name should vary per call. See section 28.9 on getting `partial()`'s fixed argument choice right.

Q27. Why does this comprehension raise a `NameError`?

```
results = [y for x in range(5) if (y := x * 2) > 4]
print(y)
```

Answer: The walrus operator (`:=`) inside the comprehension binds `y` within the comprehension's own scope in older Python versions' edge cases, but more commonly here: `y` IS accessible after the comprehension in Python 3.8+ (walrus leaks to enclosing scope) -- so this actually works in modern Python. If `NameError` occurs, it typically means the comprehension produced zero matches internally before any assignment executed; verify the condition. This question tests careful reading of walrus-in-comprehension scoping rules, an advanced edge case beyond ordinary comprehension scoping.

Q28. This code silently produces wrong totals. Find the hidden bug.

```
total = 0
def add(x):
    global total
    total += x
    return x * 2

doubled = list(map(add, [1, 2, 3]))
doubled_again = list(map(add, [1, 2, 3]))
print(total)
```

Answer: total ends up as 12, not 6, because `add()` has a hidden side effect and `map()` is called twice over the same data, silently double-counting. This is exactly the hidden-side-effect anti-pattern from section 28.5; `add()` should be pure, with any totaling done as a separate, explicit step.

31.4 'What Is the Difference Between...' Questions (29-36)

Q29. `map()` vs list comprehension?

Answer: Both transform an iterable; `map()` is lazy and takes a callable, comprehensions are eager (unless using `()`) and use inline expression syntax. Comprehensions are often more readable for simple inline transforms; `map()` is preferable with an existing named function. See section 5.12.

Q30. `filter()` vs a comprehension with `if`?

Answer: Equivalent in result; `filter()` takes a callable and is lazy, a comprehension's `if`-clause is inline and eager (for a list comprehension). See section 6.9.

Q31. lambda vs def?

Answer: lambda is a single, anonymous expression, typically used inline; def supports multiple statements, has a proper name for tracebacks, and can carry a docstring. See section 4.5.

Q32. is vs == when comparing function results, and __cause__ vs a pure function's determinism?

Answer: This tests whether the candidate conflates unrelated topics; is/== is an identity/equality distinction (see the Operators handbook) unrelated to function purity. A pure function's determinism means calling it twice with equal arguments returns EQUAL (==) results, not necessarily the SAME (is) object, especially for compound return values like a new list or dict.

Q33. reduce() vs a for loop with an accumulator?

Answer: Functionally equivalent; reduce() is more compact and expresses the 'fold to one value' intent directly once familiar, a loop is more immediately readable to a broader audience. See section 7.9.

Q34. partial() vs a closure that fixes arguments manually?

Answer: Both produce a specialized function with some arguments pre-filled; partial() is more declarative and slightly more efficient for this specific purpose, while a manual closure offers full custom logic beyond simple argument-fixing. See sections 3.3 and 11.2.

Q35. __context__ vs a side effect in a supposedly pure function?

Answer: __context__ is an exception-chaining concept (covered in the Exception Handling handbook) unrelated to function purity; this question tests recognizing an out-of-scope term. A side effect in a function that claims to be pure is simply a bug/violation of purity, not related to exception chaining at all.

Q36. Generator expression vs list comprehension?

Answer: A generator expression (parentheses) is lazy and memory-efficient, producing values on demand and only once; a list comprehension (square brackets) is eager, computing and storing every value immediately, and can be iterated repeatedly. See section 15.5 and 23.3.

31.5 Scenario-Based Questions (37-43)**Q37. You need to process a 50 GB CSV file that cannot fit in memory. What functional approach would you use?**

Answer: A lazy, generator-based pipeline: read the file line by line with a generator, parse and filter with generator expressions, and only materialize (list()) small final results, keeping memory usage bounded throughout. See section 23.6.

Q38. A teammate wrote a deeply nested chain of map(filter(map(...))) calls that nobody can read. What would you suggest?

Answer: Break it into named intermediate steps (either separate variables or a pipe()/compose() utility),

and consider whether comprehensions would express the same logic more directly. See sections 21 and 28.3.

Q39. You need to run the same pure transformation over 10 million independent records as fast as possible on a multi-core machine. What would you do?

Answer: Confirm the transformation is genuinely pure (no shared mutable state), then use `multiprocessing.Pool.map()` to distribute the work across processes, since Python's GIL prevents true CPU-bound parallelism via plain threads. See section 27.2.

Q40. Your team wants unit tests for a data-cleaning function that currently reads directly from a live database inside the function. What would you change first?

Answer: Refactor so the function accepts data as a parameter (pure transformation) rather than fetching it itself; keep the database read as a separate, thin, impure boundary function. This makes the core cleaning logic trivially testable. See sections 14.5 and 26.2.

Q41. You need to build a family of similarly configured API-calling functions that differ only by endpoint. What tool fits best?

Answer: `functools.partial()`, fixing the shared arguments (`base_url`, `api_key`, `headers`) once and leaving endpoint to vary per call. See section 11.5.

Q42. A `reduce()` call in production is hard for new team members to understand. What are your options?

Answer: Replace it with a specific built-in if one applies (`sum/min/max`), rewrite it as an explicit loop, or, if `reduce()` is genuinely appropriate, extract the accumulator logic into a small, well-named, documented function instead of an inline lambda. See sections 7.12 and 28.2.

Q43. You need to group log entries by hour, but the entries arrive out of order. Can you use `itertools.groupby()` directly?

Answer: No, not directly; `groupby()` only groups consecutive matching elements. Sort the entries by hour first (`sorted(entries, key=...)`), then apply `groupby()`. See section 24.8.

31.6 Data Science/AI-Oriented Questions (44-50)

Q44. Why might you choose pandas vectorized operations over `map()/lambda` for a `DataFrame` column transformation?

Answer: Pandas vectorized operations are implemented in optimized, compiled code operating on whole columns at once, and are almost always faster and more idiomatic than applying a Python-level function row by row via `map()` or `.apply(lambda ...)`. See section 17.8.

Q45. When is a lambda or named function still useful inside a pandas workflow?

Answer: Inside `df['col'].apply(func)`, for logic too complex or conditional for a simple vectorized

expression, and for preparing raw records before they become a DataFrame at all. See section 17.8.

Q46. How does lazy evaluation help when training a model on a dataset too large to fit in memory?

Answer: A lazy, generator-based data loader can stream batches of training data on demand rather than requiring the entire dataset to be loaded into memory upfront, enabling training on datasets far larger than available RAM. See sections 18.5 and 23.7.

Q47. Why is it important that a feature-engineering function used at both training and inference time be pure?

Answer: A pure feature function guarantees the exact same input always produces the exact same features, preventing subtle training/serving skew where the same raw input would otherwise be featurized differently depending on hidden state (like a global or a stale cache) at training time versus inference time. See sections 14 and 18.1.

Q48. How would you safely parallelize preprocessing of a large list of independent records for a data pipeline?

Answer: Ensure the per-record preprocessing function is pure, then use `multiprocessing.Pool.map()` (or an equivalent) to apply it across records in parallel, since purity guarantees no race conditions between concurrent calls. See section 27.2.

Q49. In an LLM pipeline, why might you use `functools.partial()` when building prompts?

Answer: To fix a shared prompt template (or other constant configuration) once, producing a specialized, reusable prompt-building function, avoiding repeated boilerplate at every call site. See section 19.6.

Q50. Why is `filter(None, map(extract_answer, completions))` a useful pattern when processing LLM outputs?

Answer: `map()` applies the extraction/cleaning logic to every raw completion, and `filter(None, ...)` then discards any completion that produced no usable answer (a falsy result, like an empty string or `None`), combining transformation and validity filtering in one concise, readable pipeline. See section 19.5.

32. Practical Exercises

40 exercises, progressing from beginner to expert. Attempt each one yourself before turning to the Solutions Section at the end of this part. Exercises are numbered E1 through E40 for easy cross-referencing with their solutions.

32.1 Beginner Exercises (E1-E10)

- E1. Use `map()` to convert a list of Celsius temperatures `[0, 20, 37, 100]` to Fahrenheit.
- E2. Use `filter()` to keep only the even numbers from `[1, 2, 3, 4, 5, 6, 7, 8]`.
- E3. Write a lambda that returns `True` if a number is negative.
- E4. Use `sorted()` with `key=` to sort `['banana', 'kiwi', 'apple']` by length.
- E5. Use `functools.reduce()` to compute the product of `[1, 2, 3, 4, 5]`.
- E6. Write a pure function `is_palindrome(s)` that returns `True` if a string reads the same forwards and backwards, ignoring case.
- E7. Use `filter(None, ...)` to remove all falsy values from `[0, 1, "", 'hi', None, [], [1]]`.
- E8. Use a list comprehension to build a list of squares of the even numbers from 1 to 20.
- E9. Write a function `make_greeter(greeting)` that returns a function taking a name and returning a greeting message (a closure).
- E10. Use `operator.itemgetter` to sort a list of (name, age) tuples by age.

32.2 Intermediate Exercises (E11-E20)

- E11. Write a higher-order function `apply_n_times(func, n, value)` that applies `func` to `value` `n` times in sequence.
- E12. Use `functools.partial()` to create a function `is_long_word(word)` fixed to check words longer than 6 characters, based on a general `is_longer_than(n, word)` function.
- E13. Given a list of dicts representing products, use `reduce()` to build a dictionary mapping category -> total stock.
- E14. Write a closure-based counter factory `make_counter(start=0)` whose returned function increments and returns the count on each call.
- E15. Use `itertools.groupby()` to group a pre-sorted list of employee dicts by department.
- E16. Write `compose(f, g)` that returns a new function equivalent to `f(g(x))`, then use it to compose `str.strip` and `str.upper`.
- E17. Use a generator expression to lazily compute the sum of squares of numbers from 1 to 1,000,000 without building a full list.
- E18. Use `functools.lru_cache` to memoize a recursive Fibonacci function, then time the difference with and without caching.
- E19. Write a function `safe_divide_all(numbers, divisor)` using `filter()` and `map()` together that divides only the numbers where the result would not raise an error.
- E20. Given a list of sentences, use `map()` and a pure function to count the number of words in each sentence.

32.3 Advanced Exercises (E21-E30)

- E21. Build a pipe(*functions) utility (left to right) and use it to compose a 4-stage text-cleaning pipeline: strip, lower, remove punctuation, collapse whitespace.
- E22. Write a decorator @timer that prints how long a function took to run, using functools.wraps to preserve metadata.
- E23. Fix the classic late-binding closure bug in this code without changing the loop structure: functions = [lambda: i for i in range(5)].
- E24. Use itertools.islice() and a generator to lazily read and print only lines 100-110 of a very large (simulated) file.
- E25. Write a pure function normalize_dataset(records, field) that min-max scales one numeric field across a list of dict records, without mutating the input.
- E26. Use functools.singledispatch to write a describe() function with different behavior for int, str, and list arguments.
- E27. Build a validation pipeline using filter() and multiple predicate functions combined with all(), rejecting any record that fails ANY predicate.
- E28. Write a reduce()-based function that flattens a list of lists of arbitrary (but not nested-within-nested) depth into one flat list.
- E29. Use itertools.product() to generate every combination of 3 hyperparameters, each with 2 possible values, as a list of dicts.
- E30. Write a retry-safe pure-computation wrapper (no actual I/O) that demonstrates why retry logic itself is inherently NOT purely functional (hint: think about what must vary between attempts).

32.4 Expert Exercises (E31-E40)

- E31. Design and implement a small functional pipeline framework: a Pipeline class supporting .add_stage(func) and .run(data), where each stage is validated to be callable at add time.
- E32. Implement a memoizing decorator from scratch (without functools.lru_cache) that supports functions with both positional and keyword arguments.
- E33. Write a function that safely composes an arbitrary list of functions, catching and reporting exactly which stage failed if an exception occurs partway through (functional style, using a Result-like dict return rather than raising).
- E34. Implement curry(func) that transforms a function taking multiple positional arguments into a chain of single-argument functions, e.g. curry(add)(1)(2) == 3.
- E35. Using multiprocessing.Pool, parallelize a pure feature-engineering function over 100,000 synthetic records and measure the speedup versus a sequential map().
- E36. Write a lazy, generator-based batch() function that yields fixed-size batches from any iterable (not just a list), and prove it works on an infinite itertools.count() stream using islice().
- E37. Build a small rule-engine using a dictionary of name -> pure predicate function, then write a function that evaluates a record against a named, composable set of rules combined with AND/OR logic passed in as configuration.

- E38. Implement your own version of `itertools.groupby()` using only a dictionary and a single pass (accepting the SORTED-input requirement, or explicitly handling unsorted input differently and documenting the difference).
- E39. Write unit tests (using `unittest` or plain `assert`) for the Data Preprocessing Pipeline project from section 30.6, covering at least one edge case per stage (all-None input, single-element input, already-scaled input).
- E40. Design a fully composed, production-style AI text pipeline (normalize -> tokenize -> filter stopwords -> lemmatize placeholder -> vectorize placeholder) using only composition utilities from this handbook, with each stage independently unit tested.

32.5 Solutions Section

Complete solutions and explanations for every exercise above, in order.

E1 Solution

```
celsius = [0, 20, 37, 100]
fahrenheit = list(map(lambda c: c * 9 / 5 + 32, celsius))
print(fahrenheit)
```

Expected Output: [32.0, 68.0, 98.6, 212.0]

E2 Solution

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)
```

Expected Output: [2, 4, 6, 8]

E3 Solution

```
is_negative = lambda x: x < 0
print(is_negative(-5), is_negative(5))
```

Expected Output: True False

E4 Solution

```
words = ['banana', 'kiwi', 'apple']
print(sorted(words, key=len))
```

Expected Output: ['kiwi', 'apple', 'banana']

E5 Solution

```
from functools import reduce
print(reduce(lambda acc, x: acc * x, [1, 2, 3, 4, 5]))
```

Expected Output: 120

E6 Solution

```
def is_palindrome(s):
    cleaned = s.lower()
    return cleaned == cleaned[::-1]
print(is_palindrome('Racecar'))
```

Expected Output / Note: True. Pure: no side effects, deterministic. See section 14.

E7 Solution

```
values = [0, 1, '', 'hi', None, [], [1]]  
print(list(filter(None, values)))
```

Expected Output: [1, 'hi', [1]]

E8 Solution

```
squares = [n ** 2 for n in range(1, 21) if n % 2 == 0]  
print(squares)
```

Expected Output: [4, 16, 36, 64, 100, 144, 196, 256, 324, 400]

E9 Solution

```
def make_greeter(greeting):  
    def greet(name):  
        return f'{greeting}, {name}!'  
    return greet  
hi = make_greeter('Hi')  
print(hi('Ada'))
```

Expected Output: Hi, Ada!

E10 Solution

```
from operator import itemgetter  
people = [('Ada', 34), ('Alan', 28), ('Grace', 41)]  
print(sorted(people, key=itemgetter(1)))
```

Expected Output: [('Alan', 28), ('Ada', 34), ('Grace', 41)]

E11 Solution

```
def apply_n_times(func, n, value):  
    for _ in range(n):  
        value = func(value)  
    return value  
print(apply_n_times(lambda x: x * 2, 3, 1))
```

Expected Output: 8 (1 -> 2 -> 4 -> 8)

E12 Solution

```
from functools import partial  
def is_longer_than(n, word):  
    return len(word) > n  
is_long_word = partial(is_longer_than, 6)  
print(is_long_word('functional'))
```

Expected Output: True

E13 Solution

```
from functools import reduce
products = [{'category': 'A', 'stock': 5}, {'category': 'B', 'stock': 2},
{'category': 'A', 'stock': 3}]
totals = reduce(lambda acc, p: {**acc, p['category']: acc.get(p['category'], 0) +
p['stock']}, products, {})
print(totals)
```

Expected Output: {'A': 8, 'B': 2}

E14 Solution

```
def make_counter(start=0):
    count = start
    def counter():
        nonlocal count
        count += 1
        return count
    return counter
c = make_counter(10)
print(c(), c(), c())
```

Expected Output: 11 12 13

E15 Solution

```
from itertools import groupby
employees = sorted(
    [{'name': 'Ada', 'dept': 'Eng'}, {'name': 'Alan', 'dept': 'Sales'}, {'name':
'Grace', 'dept': 'Eng'}],
    key=lambda e: e['dept']
)
for dept, group in groupby(employees, key=lambda e: e['dept']):
    print(dept, [e['name'] for e in group])
```

Expected Output: Eng ['Ada', 'Grace'] then Sales ['Alan']. Note: sorting first was required, per section 24.8.

E16 Solution

```
def compose(f, g):
    return lambda x: f(g(x))
clean = compose(str.upper, str.strip)
print(clean(' hello '))
```

Expected Output: HELLO

E17 Solution

```
total = sum(n ** 2 for n in range(1, 1_000_001))
print(total)
```

Expected Output / Note: 333333833333500000. Using a generator expression (parentheses, not brackets) avoids building a million-element list in memory. See section 15.5 and 23.3.

E18 Solution

```
from functools import lru_cache
import time

@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

start = time.time()
fib(35)
print(time.time() - start) # fast, milliseconds, due to caching
```

Explanation: Without `@lru_cache`, `fib(35)` takes seconds due to redundant recomputation; with it, cached sub-results make it near-instant. See section 9.5.

E19 Solution

```
def safe_divide_all(numbers, divisor):
    if divisor == 0:
        return []
    return list(map(lambda n: n / divisor, numbers))
print(safe_divide_all([10, 20, 30], 5))
print(safe_divide_all([10, 20], 0))
```

Expected Output: [2.0, 4.0, 6.0] then []

E20 Solution

```
sentences = ['hello world', 'functional programming is fun']
word_counts = list(map(lambda s: len(s.split()), sentences))
print(word_counts)
```

Expected Output: [2, 4]

E21 Solution

```
import re
from functools import reduce
def pipe(*functions):
    return lambda x: reduce(lambda acc, f: f(acc), functions, x)

clean = pipe(
    str.strip,
    str.lower,
    lambda s: re.sub(r'^\w\s', '', s),
    lambda s: ' '.join(s.split()),
)
print(clean(' Hello, World!! '))
```

Expected Output: hello world

E22 Solution

```
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f'{func.__name__} took {time.time() - start:.4f}s')
        return result
    return wrapper

@timer
def slow_square(x):
    time.sleep(0.1)
    return x ** 2

slow_square(5)
```

Explanation: Prints 'slow_square took 0.10..s'; @wraps ensures slow_square.__name__ still correctly reports 'slow_square', not 'wrapper'. See sections 9.4 and 20.5.

E23 Solution

```
functions = [lambda i=i: i for i in range(5)]
print([f() for f in functions])
```

Expected Output: [0, 1, 2, 3, 4]. The default argument i=i captures the current value at lambda-creation time. See section 28.8.

E24 Solution

```
from itertools import islice
def line_generator(n_lines):
    for i in range(n_lines):
        yield f'line {i}'

lines_100_to_110 = list(islice(line_generator(1000), 100, 111))
print(lines_100_to_110)
```

Expected Output: ['line 100', 'line 101', ..., 'line 110'], read lazily -- islice(iterable, start, stop) supports a start offset directly. See section 24.6.

E25 Solution

```
def normalize_dataset(records, field):
    values = [r[field] for r in records]
    lo, hi = min(values), max(values)
```

```

    return [{**r, field: (r[field] - lo) / (hi - lo)} for r in records]

data = [{'score': 10}, {'score': 50}, {'score': 90}]
print(normalize_dataset(data, 'score'))

```

Expected Output: [{'score': 0.0}, {'score': 0.5}, {'score': 1.0}]. Original list left untouched.

E26 Solution

```

from functools import singledispatch
@singledispatch
def describe(x):
    return f'unknown: {x}'
@describe.register
def _(x: int):
    return f'int: {x}'
@describe.register
def _(x: str):
    return f'str of length {len(x)}'
@describe.register
def _(x: list):
    return f'list with {len(x)} items'
print(describe(5), describe('hi'), describe([1,2,3]))

```

Expected Output: int: 5 str of length 2 list with 3 items

E27 Solution

```

def build_validator(*predicates):
    return lambda record: all(p(record) for p in predicates)

is_valid = build_validator(
    lambda r: r.get('age', 0) > 0,
    lambda r: r.get('name'),
)
records = [{'name': 'Ada', 'age': 30}, {'name': '', 'age': 30}, {'name': 'Alan',
'age': -1}]
print(list(filter(is_valid, records)))

```

Expected Output: [{'name': 'Ada', 'age': 30}]

E28 Solution

```

from functools import reduce
nested = [[1, 2], [3, 4], [5]]
flat = reduce(lambda acc, lst: acc + lst, nested, [])
print(flat)

```

Expected Output: [1, 2, 3, 4, 5]

E29 Solution

```

from itertools import product

```

```
grid = list(product([0.01, 0.1], [10, 20], [True, False]))
configs = [{'lr': lr, 'batch': b, 'shuffle': s} for lr, b, s in grid]
print(len(configs))
```

Expected Output: 8 (2 x 2 x 2 combinations)

E30 Solution

```
# Retry logic is inherently NOT purely functional: it depends on WHEN it is
# called (timing) and typically on an operation that can genuinely produce
# different results on different attempts (e.g. a flaky network call).
# A 'pure' retry wrapper around a genuinely pure function is pointless,
# since a pure function would fail identically every single time.
def demonstrate():
    return 'Retry logic only makes sense around IMPURE operations.'
print(demonstrate())
```

Explanation: This exercise is conceptual: it reinforces that retry logic (see the Exception Handling handbook) is meaningful specifically for impure, potentially-flaky operations, not for pure computations, which would simply fail the same way every retry.

E31 Solution

```
class Pipeline:
    def __init__(self):
        self.stages = []
    def add_stage(self, func):
        if not callable(func):
            raise TypeError('Stage must be callable')
        self.stages.append(func)
        return self
    def run(self, data):
        for stage in self.stages:
            data = stage(data)
        return data

p = Pipeline().add_stage(str.strip).add_stage(str.upper)
print(p.run(' hello '))
```

Expected Output: HELLO. `.add_stage()` validates callability immediately (fail-fast) and returns self to allow chaining.

E32 Solution

```
def memoize(func):
    cache = {}
    def wrapper(*args, **kwargs):
        key = (args, tuple(sorted(kwargs.items())))
        if key not in cache:
            cache[key] = func(*args, **kwargs)
        return cache[key]
    return wrapper
```

```
@memoize
def slow_add(a, b=0):
    return a + b
print(slow_add(2, b=3), slow_add(2, b=3))
```

Expected Output: 5 5 (second call served from cache). The key combines args and sorted kwargs items to handle keyword arguments consistently.

E33 Solution

```
def safe_pipe(*functions):
    def piped(x):
        for i, f in enumerate(functions):
            try:
                x = f(x)
            except Exception as e:
                return {'success': False, 'failed_stage': i, 'error': str(e)}
        return {'success': True, 'result': x}
    return piped

pipeline = safe_pipe(lambda x: x + 1, lambda x: 1 / x, lambda x: x * 2)
print(pipeline(-1))
```

Expected Output: {'success': False, 'failed_stage': 1, 'error': 'division by zero'} (since $-1 + 1 = 0$, then $1/0$ fails)

E34 Solution

```
def curry(func):
    def curried(a):
        def inner(b):
            return func(a, b)
        return inner
    return curried

add = lambda a, b: a + b
curried_add = curry(add)
print(curried_add(1)(2))
```

Expected Output: 3. This simplified curry() supports exactly two arguments; a general-purpose curry would use *args and check arity, an extension worth exploring beyond this exercise.

E35 Solution

```
from multiprocessing import Pool
import time

def featurize(record):
    return {'**record', 'squared': record['value'] ** 2}

records = [{'value': i} for i in range(100_000)]

start = time.time()
```

```

sequential = list(map(featurize, records))
seq_time = time.time() - start

start = time.time()
with Pool(4) as pool:
    parallel = pool.map(featurize, records)
par_time = time.time() - start

print(seq_time, par_time)

```

Explanation: Exact numbers vary by machine, but for a cheap per-record operation like this, parallel overhead (process startup, data serialization) often makes it SLOWER than sequential map(); multiprocessing pays off for genuinely expensive per-record work, an important, honest lesson connecting to section 27.4's caveats.

E36 Solution

```

from itertools import count, islice

def batch(iterable, size):
    it = iter(iterable)
    while True:
        chunk = list(islice(it, size))
        if not chunk:
            return
        yield chunk

infinite_batches = batch(count(1), 3)
print(list(islice(infinite_batches, 2)))

```

Expected Output: [[1, 2, 3], [4, 5, 6]]. batch() itself is lazy and works correctly even on an infinite stream, since it only ever pulls as many items as the caller actually consumes.

E37 Solution

```

RULES = {
    'is_adult': lambda r: r['age'] >= 18,
    'has_email': lambda r: '@' in r.get('email', ''),
}

def evaluate(record, rule_names, mode='all'):
    checks = (RULES[name](record) for name in rule_names)
    return all(checks) if mode == 'all' else any(checks)

record = {'age': 20, 'email': 'ada@example.com'}
print(evaluate(record, ['is_adult', 'has_email'], mode='all'))

```

Expected Output: True. Rules are named, composable, pure predicates looked up dynamically by name (section 2.6), combined with configurable AND (all)/OR (any) logic.

E38 Solution

```

def my_groupby(iterable, key):

```

```

groups = {}
for item in iterable:
    k = key(item)
    groups.setdefault(k, []).append(item)
return groups

records = [{'dept': 'Eng'}, {'dept': 'Sales'}, {'dept': 'Eng'}]
print(my_groupby(records, key=lambda r: r['dept']))

```

Expected Output / Note: {'Eng': [{'dept': 'Eng'}, {'dept': 'Eng'}], 'Sales': [{'dept': 'Sales'}]}. Unlike `itertools.groupby()`, this version correctly groups NON-consecutive matches too, at the cost of building the full groups dictionary in memory rather than staying lazy.

E39 Solution

```

import unittest

def remove_none(values): return [v for v in values if v is not None]
def impute_mean(values):
    valid = remove_none(values)
    mean = sum(valid) / len(valid) if valid else 0
    return [v if v is not None else mean for v in values]
def min_max_scale(values):
    lo, hi = min(values), max(values)
    return [0.5] * len(values) if hi == lo else [(v - lo) / (hi - lo) for v in values]

class TestPipeline(unittest.TestCase):
    def test_all_none(self):
        self.assertEqual(impute_mean([None, None]), [0, 0])
    def test_single_element(self):
        self.assertEqual(min_max_scale([5]), [0.5])
    def test_already_scaled(self):
        self.assertEqual(min_max_scale([0, 1]), [0.0, 1.0])

```

Explanation: All three tests pass; note `min_max_scale` required a guard for `hi == lo` (a single-element or constant-value list) to avoid a `ZeroDivisionError`, an edge case surfaced specifically BY writing the test. This demonstrates section 26's claim that pure functions make edge cases easy to discover and verify.

E40 Solution

```

import re
from functools import reduce

def pipe(*fns):
    return lambda x: reduce(lambda acc, f: f(acc), fns, x)

def normalize(text): return re.sub(r'[^a-z0-9\s]', '', text.lower())
def tokenize(text): return text.split()
def remove_stopwords(tokens): return [t for t in tokens if t not in {'the', 'a', 'is'}]
def lemmatize_placeholder(tokens): return tokens # placeholder for a real lemmatizer

```

```
def vectorize_placeholder(tokens): return {t: tokens.count(t) for t in set(tokens)}

text_pipeline = pipe(normalize, tokenize, remove_stopwords, lemmatize_placeholder,
vectorize_placeholder)
print(text_pipeline('The cat is on the mat, the mat is red.'))
```

Expected Output: {'cat': 1, 'on': 1, 'mat': 1, 'mat': 1, 'red': 1} (exact keys depend on punctuation handling). Each stage remains independently unit-testable exactly as in E39, and placeholder stages can later be swapped for real implementations without touching any other stage, directly demonstrating the modularity goal of section 22.4.

33. Final Cheat Sheet

33.1 Terminology

Code Line	Explanation
First-class function	A function treated as an ordinary value: assignable, passable, returnable, storable.
Higher-order function	A function that accepts and/or returns another function.
Pure function	Deterministic, no side effects; same input always gives same output.
Referential transparency	A call can be replaced by its return value with no change in behavior.
Side effect	Any observable effect beyond a function's return value (mutation, I/O, global state change).
Immutability	Data that cannot be changed after creation; 'modification' produces new data instead.
Closure	A function that remembers variables from its enclosing scope after that scope has finished.
Lazy evaluation	Deferring computation until a value is actually needed.
Composition	Combining functions so one's output feeds the next's input.
Declarative style	Describing WHAT result is wanted, not the step-by-step HOW.

33.2 Lambda Syntax

```
lambda: 'no args'
lambda x: x * 2
lambda x, y: x + y
lambda x, y=10: x + y
lambda x: 'pos' if x > 0 else 'neg'
```

33.3 map(), filter(), reduce()

```
list(map(func, iterable))
list(map(func, iter1, iter2))           # multiple iterables
list(filter(func, iterable))
list(filter(None, iterable))           # keep only truthy

from functools import reduce
reduce(func, iterable)                  # no initializer
reduce(func, iterable, initial)         # with initializer
```

33.4 partial()

```
from functools import partial
specialized = partial(func, fixed_arg1, kw=fixed_value)
```

33.5 operator Module Quick Reference

Code Line	Explanation
add, sub, mul, truediv, floordiv, mod, pow	Arithmetic operators as functions.

Code Line	Explanation
eq, ne, lt, le, gt, ge	Comparison operators as functions.
itemgetter(key_or_index)	Extracts one field/index; itemgetter(k1, k2) extracts a tuple of several.
attrgetter('name')	Extracts an object attribute.
methodcaller('name', *args)	Calls a named method on each element.

33.6 itertools Quick Reference

Code Line	Explanation
chain(a, b, ...)	Lazily concatenate iterables.
compress(data, selectors)	Keep elements where the matching selector is truthy.
count(start, step)	Infinite counter.
cycle(iterable)	Infinitely repeat an iterable.
repeat(value, n)	Repeat a value n times (or forever if n omitted).
islice(iterable, stop) / (iterable, start, stop)	Lazily slice an iterator.
takewhile(cond, it) / dropwhile(cond, it)	Take/skip a leading run while cond holds.
groupby(it, key)	Group CONSECUTIVE matching elements; sort first!
product(a, b)	Cartesian product.
permutations(it, r)	Ordered selections of length r.
combinations(it, r)	Unordered selections of length r.

33.7 Closures

```
def outer(x):
    def inner(y):
        nonlocal x    # only needed to REASSIGN x, not to read it
        return x + y
    return inner
```

33.8 Pure Functions Checklist

- Same input always produces the same output.
- No mutation of arguments or external/global state.
- No I/O (print, file, network, database) inside the function.
- No dependency on randomness, the clock, or other hidden external state.

33.9 Function Composition

```
def compose(f, g):
    return lambda x: f(g(x))           # right to left: f(g(x))

from functools import reduce
def pipe(*fns):
    return lambda x: reduce(lambda acc, f: f(acc), fns, x)  # left to right
```

33.10 Comprehensions

```
[expr for x in it]           # list
[expr for x in it if cond]   # filtered list
{expr for x in it}          # set
{k: v for x in it}          # dict
(expr for x in it)           # generator (lazy)
[expr for row in matrix for expr in row] # flatten
```

33.11 Lazy Evaluation Quick Facts

- `map()`, `filter()`, and `()` generator expressions are all lazy; `[]` comprehensions are eager.
- A lazy object (`map/filter/generator`) can be iterated only ONCE, then it is exhausted.
- Wrap in `list()/set()/tuple()` to force full, repeatable materialization.
- Prefer laziness for large or streaming data to keep memory usage bounded.

33.12 Best Practices Summary

Code Line	Explanation
Use <code>def</code>	Multi-statement logic, reused functions, public APIs.
Use <code>lambda</code>	Trivial, single-use, inline expressions only.
Use comprehensions	Default choice for simple transform/filter into a new collection.
Use <code>map()/filter()</code>	When the function is already named and reused.
Use <code>reduce()</code>	Only when <code>sum/min/max/any/all</code> do not already cover it.
Use generators	Large/streaming data, or early-exit computations.
Use <code>partial()</code>	Genuinely constant arguments shared across many calls.
Use <code>composition/pipe()</code>	A clear, reusable, ordered sequence of pure transformation stages.
Golden rule	Prioritize Pythonic readability over dogmatic functional 'purity' for its own sake.

End of Document

Kajola Gbenga | Data Scientist & AI Engineer